M02 — Prácticas integrales: programar y probar APIs REST con .NET 8

Autor: JAIME GALLO

Metodología de trabajo

Cada apartado parte del diseño REST explicado en teoría y culmina en **código ejecutable y peticiones HTTP**. Se trabaja con Visual Studio, ASP.NET Core 8 Minimal API, EF Core 8 y SQL Server Express LocalDB. Los cuatro estados de referencia están en `PROYECTO/2.1` a `PROYECTO/2.4`: no son simulaciones, sino proyectos con endpoints reales probados por HTTP.

La actividad práctica no requiere operaciones Git. El alumnado construye primero la etapa 2.1, incorpora los verbos HTTP en 2.2, conecta la validación en 2.3 y documenta el mismo servicio en 2.4. Para contrastar el resultado, puede abrir por separado los proyectos de referencia. La copia no sustituye la escritura y comprensión de código.

Persistencia: la API del alumno utiliza SQL Server Express LocalDB. Los tests de integración usan EF Core InMemory únicamente para ejecutarse de forma independiente en GitHub Actions. `EnsureCreated` se emplea en desarrollo para simplificar el arranque del ejercicio; no se propone para despliegues productivos.

Punto 2.1 — Del diseño REST a los primeros endpoints GET

Proyecto comprobado: `PROYECTO/2.1-diseno-apis-rest`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.1.1 — Crear el proyecto ASP.NET Core Empty desde Visual Studio

Objetivo del paso. Crear el proyecto ASP.NET Core Empty desde Visual Studio.

Acciones guiadas. Archivo > Nuevo > Proyecto > ASP.NET Core Empty. Selecciona C#, .NET 8 y el nombre AceriaApp.Api. No marques autenticación.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El programa aún se está preparando: comprobar SDK, arranque y URI antes de introducir persistencia permite aislar errores de herramienta frente a errores de API.

Verificación concreta. Verifica el TargetFramework net8.0 y arranca el proyecto vacío.

Comprobación adicional guiada. En Visual Studio crea una aplicación ASP.NET Core Empty con el framework .NET 8 y nombre AceriaApp.Api. No selecciones Web API con controladores: se utilizará Minimal API. Abre el proyecto, comprueba la versión de TargetFramework en el archivo csproj y ejecuta una compilación inicial. La primera evidencia es que el proyecto vacío compila; todavía no debe haber entidades ni endpoints de órdenes.

Si algo falla. Si falla el inicio, revisa que hayas seleccionado ASP.NET Core Empty y net8.0; no es necesario instalar controladores.

PASO 2.1.2 — Explorar Program.cs, Kestrel y el perfil de arranque

Objetivo del paso. Explorar Program.cs, Kestrel y el perfil de arranque.

Acciones guiadas. Abre Program.cs y localiza CreateBuilder, Build y Run. Ejecuta con F5, observa la dirección del perfil local y detén Kestrel.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El programa aún se está preparando: comprobar SDK, arranque y URI antes de introducir persistencia permite aislar errores de herramienta frente a errores de API.

Verificación concreta. Kestrel escucha en el puerto anunciado; no es una prueba de base de datos.

Comprobación adicional guiada. Localiza la creación de WebApplicationBuilder, builder.Build y app.Run. Coloca un punto de interrupción en la configuración de Program.cs y observa que se ejecuta al arrancar el host, antes de recibir peticiones. Identifica el puerto HTTP/HTTPS del perfil de Visual Studio y anótalo para el archivo AceriaApp.http. No atribuyas al navegador la creación de Kestrel.

Si algo falla. Si falla el inicio, revisa que hayas seleccionado ASP.NET Core Empty y net8.0; no es necesario instalar controladores.

PASO 2.1.3 — Dibujar los recursos y decidir las dos URIs GET

Objetivo del paso. Dibujar los recursos y decidir las dos URIs GET.

Acciones guiadas. Diseña las rutas GET /api/v1/ordenes y GET /api/v1/ordenes/{id:guid}. Explica qué recurso identifica cada una.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El programa aún se está preparando: comprobar SDK, arranque y URI antes de introducir persistencia permite aislar errores de herramienta frente a errores de API.

Verificación concreta. Las dos rutas usan sustantivos en plural, y el detalle tiene un Guid.

Comprobación adicional guiada. Escribe en tu cuaderno los nombres de la colección y del recurso individual. Formula dos peticiones: GET /api/v1/ordenes y GET /api/v1/ordenes/{id}. Justifica por qué la primera no necesita un identificador y por qué la segunda sí. No introduzcas aún rutas con verbos como obtenerOrden; en la aplicación se combinan nombres de recursos con métodos HTTP.

Si algo falla. Si falla el inicio, revisa que hayas seleccionado ASP.NET Core Empty y net8.0; no es necesario instalar controladores.

PASO 2.1.4 — Crear carpeta Modelo y entidad Orden

Objetivo del paso. Crear carpeta Modelo y entidad Orden.

Acciones guiadas. Añade la carpeta Modelo. Crea Orden.cs con Guid Id, Cliente, Material, Cantidad, Estado y FechaCreacion; compara con la clase de referencia.

Código real contrastado: **`src/AceriaApp.Api/Modelo/Orden.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.4
public sealed class Orden
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Cliente { get; set; } = string.Empty;
    public string Material { get; set; } = string.Empty;
    public int Cantidad { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public DateTime FechaCreacion { get; set; } = DateTime.UtcNow;
```

```
}
// FIN PASO 2.1.4
```

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. La clase Orden compila, y el estado inicial es Pendiente.

Comprobación adicional guiada. Crea Modelo/Orden.cs y declara exactamente Id Guid, Cliente string, Material string, Cantidad int, Estado string y FechaCreacion DateTime. Asigna Guid.NewGuid a la clave, Pendiente como estado inicial y DateTime.UtcNow como momento de creación. Compila. Explica qué propiedades se generan en el servidor y por qué el cliente no debería imponer su propio Estado al crear una orden.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.5 — Explicar ID Guid, Estado y FechaCreacion de la entidad

Objetivo del paso. Explicar ID Guid, Estado y FechaCreacion de la entidad.

Acciones guiadas. En Orden.cs observa que Id se genera por defecto y FechaCreacion emplea UtcNow. Distingue los datos que proporciona el servidor de los del cliente.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. No se reciben el Id ni la fecha desde una petición de creación.

Comprobación adicional guiada. Construye dos órdenes en el depurador y compara sus Id: aunque compartan Cliente y Material, son entidades distintas. Revisa que Estado comience en Pendiente y que FechaCreacion utilice UTC. Cambiar cualquiera de esos valores iniciales afectará a las respuestas posteriores; comprueba que no has puesto una fecha fija tomada del manual ni reutilizado un Guid de otra ejecución.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.6 — Crear OrdenDto como contrato de salida

Objetivo del paso. Crear OrdenDto como contrato de salida.

Acciones guiadas. En Modelo crea OrdenDto.cs como record público con los datos que va a serializar JSON; no lo conviertas en entidad de EF Core.

Código real contrastado: **`src/AceriaApp.Api/Modelo/OrdenDto.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.5
public sealed record OrdenDto(Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion)
{
    public static OrdenDto Desde(Orden x) =>
        new(x.Id,x.Cliente,x.Material,x.Cantidad,x.Estado,x.FechaCreacion);
}
// FIN PASO 2.1.5
```

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. El compilador distingue Orden de OrdenDto.

Comprobación adicional guiada. Crea el record OrdenDto con los seis campos que realmente deben salir en JSON. Compara su declaración con la entidad Orden. Señala que OrdenDto es un contrato de representación, no una tabla ni un DbSet. Si en el futuro Orden incorporase información interna, no añadas automáticamente esa propiedad al DTO. Guarda y comprueba que el tipo compila con el namespace Modelo.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.7 — Programar la conversión OrdenDto.Desde

Objetivo del paso. Programar la conversión OrdenDto.Desde.

Acciones guiadas. Añade a OrdenDto un método estático Desde(Orden x); construye el record usando las propiedades de una orden.

Código real contrastado: `src/AceriaApp.Api/Modelo/OrdenDto.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.5
public sealed record OrdenDto(Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion)
{
    public static OrdenDto Desde(Orden x) =>
        new(x.Id,x.Cliente,x.Material,x.Cantidad,x.Estado,x.FechaCreacion);
}
// FIN PASO 2.1.5
```

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. El DTO contiene el mismo Id y datos públicos que la entidad.

Comprobación adicional guiada. Programa el método estático OrdenDto.Desde para recibir una Orden y construir el record con Id, Cliente, Material, Cantidad, Estado y FechaCreacion en el orden declarado. Revisa cuidadosamente el orden de argumentos: dos strings intercambiados pueden compilar pero producir una representación incorrecta. Utiliza una orden conocida para inspeccionar el resultado y confirma que el método no accede por sí solo a la base.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.8 — Instalar EF Core 8 para SQL Server

Objetivo del paso. Instalar EF Core 8 para SQL Server.

Acciones guiadas. Desde Administrar paquetes NuGet instala Microsoft.EntityFrameworkCore.SqlServer 8.x en AceriaApp.Api, o utiliza dotnet add package con versión 8.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. La compilación reconoce UseSqlServer.

Comprobación adicional guiada. Desde el administrador de paquetes NuGet añade la versión 8 de Microsoft.EntityFrameworkCore.SqlServer al proyecto API. Comprueba que la restauración termina correctamente y que UseSqlServer aparece disponible al importar Microsoft.EntityFrameworkCore. Si el paquete corresponde a otra versión mayor del framework, corrige su referencia antes de continuar. El objetivo es habilitar el proveedor SQL Server, no instalar SQLite.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.9 — Crear AceriaDbContext y DbSet

Objetivo del paso. Crear AceriaDbContext y DbSet.

Acciones guiadas. Crea Datos/AceriaDbContext.cs; haz que herede de DbContext y declara DbSet Ordenes.

Código real contrastado: `src/AceriaApp.Api/Datos/AceriaDbContext.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Api.Datos;
// INICIO PASO 2.1.6
public sealed class AceriaDbContext(DbContextOptions<AceriaDbContext> options) :
DbContext(options)
{
    public DbSet<Orden> Ordenes => Set<Orden>();
    protected override void OnModelCreating(ModelBuilder b)
    {
        var e=b.Entity<Orden>();
        e.HasKey(x=>x.Id);
        e.Property(x=>x.Cliente).HasMaxLength(100).IsRequired();
        e.Property(x=>x.Material).HasMaxLength(100).IsRequired();
        e.Property(x=>x.Estado).HasMaxLength(24).IsRequired();
    }
}
// FIN PASO 2.1.6
```

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. Al compilar, el contexto conoce Orden y puede consultarla.

Comprobación adicional guiada. Crea Datos/AceriaDbContext.cs y deriva de DbContext. Declara el constructor que recibe DbContextOptions y expone DbSet Ordenes. La entidad se conoce a través del namespace Modelo. Compila antes de escribir OnModelCreating: de este modo puedes distinguir un problema del tipo DbContext de un error posterior en las restricciones del modelo.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.10 — Configurar OnModelCreating, clave y longitudes

Objetivo del paso. Configurar OnModelCreating, clave y longitudes.

Acciones guiadas. En OnModelCreating configura HasKey(Id), las longitudes de Cliente y Material y los campos obligatorios.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. El modelo restringe tamaños de columna y reconoce la clave.

Comprobación adicional guiada. En OnModelCreating utiliza la entidad Orden y configura HasKey para Id, MaximumLength(100) para Cliente y Material, MaximumLength(24) para Estado y Required para esos strings. Compara los tamaños definidos con las reglas que se introducirán en 2.3. No confundas esa configuración de persistencia con un mensaje de validación HTTP: un límite SQL mal aplicado puede generar un error de base de datos, no el ValidationProblem deseado.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.11 — Configurar la cadena de conexión a LocalDB

Objetivo del paso. Configurar la cadena de conexión a LocalDB.

Acciones guiadas. En appsettings.json añade ConnectionStrings:AceriaApp con (localdb)\MSSQLLocalDB y Database=AceriaAppM02. No sustituyas LocalDB por SQLite.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. La cadena se lee desde Configuration y apunta al servidor local.

Comprobación adicional guiada. En appsettings.json configura ConnectionStrings:AceriaApp para la instancia (localdb)\MSSQLLocalDB del entorno Windows. No incluyas contraseñas personales en el documento ni copies una ruta de base física de otra máquina. Utiliza la configuración que lee GetConnectionString en Program.cs. Si Visual Studio no encuentra LocalDB, comprueba la instalación de SQL Server Express LocalDB y el nombre de la instancia.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.12 — Registrar AceriaDbContext mediante AddDbContext

Objetivo del paso. Registrar AceriaDbContext mediante AddDbContext.

Acciones guiadas. En Program.cs, antes de builder.Build, añade AddDbContext con UseSqlServer y GetConnectionString.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.1.7: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));

var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
if(app.Environment.IsDevelopment()&&!
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await
scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}
```

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. El manejador HTTP podrá recibir AceriaDbContext por inyección.

Comprobación adicional guiada. Antes de builder.Build registra AddDbContext con options.UseSqlServer y GetConnectionString de AceriaApp. Coloca un punto de interrupción dentro de MapGet cuando ya exista y observa que ASP.NET Core recibe el contexto sin escribir new AceriaDbContext en el delegado. Si aparece una excepción de servicios no registrados, revisa el orden del registro y los namespaces.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

Caso práctico completo — Del navegador hasta una consulta EF Core

Este caso demuestra el trayecto completo de una petición GET antes de añadir pruebas. Abre `src/AceriaApp.Api/Program.cs` y coloca un punto de interrupción justo dentro de `app.MapGet("/api/v1/ordenes", ...)`. Ejecuta la API con F5 usando el perfil Development y abre `AceriaApp.http`. Envía `GET /api/v1/ordenes`. Cuando Visual Studio se detenga, inspecciona el argumento `AceriaDbContext db`. No lo ha creado el cliente HTTP: lo resolvió el contenedor de dependencias a partir de `AddDbContext`, configurado antes de construir la aplicación.

Continúa paso por paso. `db.Ordenes` representa la tabla de órdenes; `AsNoTracking()` indica que EF Core no va a mantener seguimiento de cambios en las entidades recuperadas. Esta opción resulta apropiada cuando no se pretende modificar esas mismas entidades. `OrderBy(x => x.FechaCreacion)` establece un orden reproducible de presentación, y `ToListAsync` ejecuta la consulta. Si no hay filas, el resultado será una lista vacía, que debe traducirse en un estado 200 con `[]` en el cuerpo JSON.

La llamada `OrdenDto.Desde` realiza el límite entre persistencia y representación. El cliente no recibe una instancia del contexto ni detalles sobre una tabla; recibe un objeto JSON con los campos públicos de la orden. La ruta de detalle añade una comprobación: si el Guid es válido pero la consulta no encuentra ninguna fila, el servidor responde 404. Comprueba esta situación con el identificador inexistente del archivo HTTP. Si recibes una excepción de conexión en lugar de 200 o 404, revisa que LocalDB está instalado y que la cadena de conexión apunta a `(localdb)\MSSQLLocalDB`.

Demostración de aprendizaje: explica por qué `Results.Ok` y `Results.NotFound` pertenecen al contrato HTTP y no al modelo de dominio. Cambia temporalmente una ruta en Program.cs y observa cómo deja de coincidir la petición anterior; restaura el código y vuelve a comprobarlo.

PASO 2.1.13 — Crear la base didáctica solo en Development

Objetivo del paso. Crear la base didáctica solo en Development.

Acciones guiadas. En Development crea la base de enseñanza con Database.EnsureCreatedAsync usando un scope; deja escrito que en producción se usarían migraciones.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. Al arrancar en Windows aparece la base AceriaAppM02 si LocalDB está instalado.

Comprobación adicional guiada. Activa EnsureCreatedAsync solo en Development y condiciona su ejecución a SkipDatabaseInitialization. Crea un scope con app.Services.CreateScope y recupera el DbContext. Ejecuta el arranque dos veces sin borrar la base y comprueba que la inicialización didáctica no es una migración de producción. En tests, la factoría desactiva este camino y usa una base InMemory aislada.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.14 — Escribir el endpoint GET de colección

Objetivo del paso. Escribir el endpoint GET de colección.

Acciones guiadas. En Program.cs escribe MapGet de /api/v1/ordenes y recibe AceriaDbContext como argumento del lambda.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.1.8: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")

;
```

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. El endpoint queda registrado y responde a GET.

Comprobación adicional guiada. Añade MapGet sobre /api/v1/ordenes con dos argumentos inyectados: AceriaDbContext y CancellationToken. Obtén db.Ordenes y prepara una consulta asíncrona. Al compilar, verifica que MapGet queda antes de app.Run y que el endpoint tiene nombre ListarOrdenes. No crees por accidente una segunda ruta distinta para el mismo recurso.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.15 — Utilizar AsNoTracking y proyectar a DTO

Objetivo del paso. Utilizar AsNoTracking y proyectar a DTO.

Acciones guiadas. Dentro de MapGet consulta db.Ordenes.AsNoTracking, materializa la lista y aplica OrdenDto.Desde a cada entidad.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. La salida expone DTOs y no instancias de la entidad EF Core.

Comprobación adicional guiada. Encadena AsNoTracking, OrderBy por FechaCreacion y ToListAsync(ct) antes de aplicar OrdenDto.Desde. Explica en voz alta qué trabajo se realiza en SQL y qué conversión sucede después de materializar la lista. No añadas Skip/Take como si el proyecto ya ofreciera paginación: aún no existe esa funcionalidad. Comprueba que el resultado se entrega con Results.Ok.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.16 — Ejecutar GET y verificar 200 con lista vacía

Objetivo del paso. Ejecutar GET y verificar 200 con lista vacía.

Acciones guiadas. Abre AceriaApp.http, ejecuta GET de colección y comprueba el código y el JSON aunque la tabla esté vacía.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. La respuesta es 200 y un arreglo JSON; no 404.

Comprobación adicional guiada. Con una base nueva sin órdenes envía GET /api/v1/ordenes y verifica dos elementos: el status 200 y un cuerpo de lista JSON vacío. Una colección existente con cero elementos no debe convertirse en 404. Si el servidor responde 500, consulta la excepción y la conexión; si devuelve 404, revisa si has registrado correctamente la ruta de colección.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.17 — Escribir el GET individual con

Objetivo del paso. Escribir el GET individual con {id:guid}.

Acciones guiadas. Añade MapGet /api/v1/ordenes/{id:guid} y recibe Guid id y DbContext inyectado.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.1.9: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")

;

app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. La ruta coincide con un identificador Guid válido.

Comprobación adicional guiada. Registra el GET individual con la plantilla /api/v1/ordenes/{id:guid}. El argumento Guid id debe corresponder al parámetro de ruta. Ejecuta una petición con un Guid sintácticamente válido que todavía no exista y observa que la búsqueda devuelve null. La restricción guid controla el emparejamiento de ruta; no verifica que la orden esté guardada.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.18 — Devolver NotFound cuando no existe la orden

Objetivo del paso. Devolver NotFound cuando no existe la orden.

Acciones guiadas. Busca mediante FirstOrDefaultAsync y devuelve Results.NotFound cuando la orden no existe; usa Results.Ok cuando aparece.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Un GUID nuevo devuelve 404 sin excepción 500.

Comprobación adicional guiada. Introduce la selección entre Results.NotFound y Results.Ok(OrdenDto.Desde(orden)). Antes de hacer la conversión comprueba orden is null, porque de lo contrario intentarías leer las propiedades de una entidad inexistente. Ejecuta de nuevo la solicitud con Guid ausente y confirma el 404. Identifica la instrucción exacta que decide el estado HTTP, en lugar de atribuir esa decisión a EF Core.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.19 — Comprobar diferencias entre ruta incorrecta y recurso ausente

Objetivo del paso. Comprobar diferencias entre ruta incorrecta y recurso ausente.

Acciones guiadas. Envía una ruta con identificador que no es GUID y otra con GUID inexistente; anota qué parte resuelve el enrutador y cuál el manejador.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Comprendes la diferencia entre plantilla que no coincide y recurso ausente.

Comprobación adicional guiada. Prueba una ruta con un segmento que no puede interpretarse como Guid y otra con un Guid válido sin fila. Ambas pueden devolver 404, pero la primera no entra al endpoint restringido mientras que la segunda ejecuta la consulta. Coloca un breakpoint en el manejador y compara qué peticiones lo alcanzan. Este diagnóstico evita que se investigue la base de datos cuando el fallo estaba en la URI.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.20 — Crear AceriaApp.http y enviar peticiones desde Visual Studio

Objetivo del paso. Crear AceriaApp.http y enviar peticiones desde Visual Studio.

Acciones guiadas. En Visual Studio abre o crea AceriaApp.http, define @Host y dos peticiones GET con la ruta versionada.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Puedes ejecutar ambos GET sin copiar URLs del navegador.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

Comprobación guiada del archivo HTTP. Configura Host con la dirección y puerto reales mostrados por Visual Studio. Ejecuta primero GET /api/v1/ordenes y observa el 200 y el array JSON, aunque esté vacío. Ejecuta después GET /api/v1/ordenes/00000000-0000-0000-0000-000000000001: una base sin esa clave devuelve 404. Sustituye finalmente el Guid por un texto inválido y explica por qué un 404 de enrutamiento no es una avería de LocalDB. Anota qué mensaje corresponde al endpoint y cuál al enrutador.

PASO 2.1.21 — Crear el proyecto xUnit de pruebas HTTP

Objetivo del paso. Crear el proyecto xUnit de pruebas HTTP.

Acciones guiadas. Crea un proyecto xUnit AceriaApp.Api.Tests y referencia AceriaApp.Api; instala Microsoft.AspNetCore.Mvc.Testing 8.

Código real contrastado: `tests/AceriaApp.Api.Tests/ApiFactory.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Datos;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.12: HTTP real; base en memoria SOLO en tests automatizados.
public sealed class ApiFactory:WebApplicationFactory<Program>
{
    private readonly string _databaseName="Tests-"+Guid.NewGuid();
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Development");
        builder.UseSetting("SkipDatabaseInitialization","true");
        builder.ConfigureTestServices(services=>
        {
            services.RemoveAll<DbContextOptions<AceriaDbContext>>();
            services.AddDbContext<AceriaDbContext>(opt=>
                opt.UseInMemoryDatabase(_databaseName));
        });
    }
}
```

Por qué funciona así. WebApplicationFactory prueba la aplicación como servidor HTTP: permite distinguir compilación C# de comportamiento observable del enrutamiento y la serialización.

Verificación concreta. Las clases de tests referencian Program de la API.

Comprobación adicional guiada. Crea el proyecto de pruebas AceriaApp.Api.Tests con xUnit y referencia el proyecto API. Añade Microsoft.AspNetCore.Mvc.Testing compatible con .NET 8. Ejecuta dotnet test o el Explorador de pruebas y verifica que la factoría puede iniciarse. Un proyecto de pruebas que solo compila aún no demuestra el contrato GET; en pasos posteriores se enviarán peticiones reales mediante HttpClient.

Si algo falla. Si falla ApiFactory, comprueba Program partial y el registro de DbContext con InMemory solo para tests.

PASO 2.1.22 — Configurar WebApplicationFactory y base aislada para tests

Objetivo del paso. Configurar WebApplicationFactory y base aislada para tests.

Acciones guiadas. Crea ApiFactory derivada de WebApplicationFactory; sustituye solo para pruebas el proveedor por EF Core InMemory.

Código real contrastado: `tests/AceriaApp.Api.Tests/ApiFactory.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Datos;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
```

```
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.12: HTTP real; base en memoria SOLO en tests automatizados.
public sealed class ApiFactory:WebApplicationFactory<Program>
{
    private readonly string _databaseName="Tests-"+Guid.NewGuid();
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Development");
        builder.UseSetting("SkipDatabaseInitialization","true");
        builder.ConfigureTestServices(services=>
        {
            services.RemoveAll<DbContextOptions<AceriaDbContext>>();
            services.AddDbContext<AceriaDbContext>(opt=>
                opt.UseInMemoryDatabase(_databaseName));
        });
    }
}
```

Por qué funciona así. WebApplicationFactory prueba la aplicación como servidor HTTP: permite distinguir compilación C# de comportamiento observable del enrutamiento y la serialización.

Verificación concreta. Las pruebas arrancan ASP.NET Core en memoria; la aplicación real conserva LocalDB.

Comprobación adicional guiada. Configura ApiFactory para heredar de WebApplicationFactory y sustituir el DbContext por el proveedor InMemory exclusivo del test. Asegúrate de desactivar la inicialización real de LocalDB en esa factoría. Al ejecutar los tests no necesitas una instancia SQL Server en el agente, pero la configuración normal de la API sigue siendo SQL Server. Comprueba que las pruebas no comparten datos entre ejecuciones.

Si algo falla. Si falla ApiFactory, comprueba Program partial y el registro de DbContext con InMemory solo para tests.

PASO 2.1.23 — Probar 200, 404 y DTO mediante GetTests

Objetivo del paso. Probar 200, 404 y DTO mediante GetTests.

Acciones guiadas. Crea GetTests y tres casos: colección 200, GUID inexistente 404, orden insertada recuperada como OrdenDto.

Código real contrastado: `tests/AceriaApp.Api.Tests/GetTests.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.Extensions.DependencyInjection;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.13: dos peticiones HTTP, código y DTO.
public class GetTests
{
    [Fact] public async Task ListaVacia_Es200()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes");
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
    }
    [Fact] public async Task RecursoInexistente_Es404()
```

```
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+Guid.NewGuid());
        Assert.Equal(HttpStatusCode.NotFound,r.StatusCode);
    }
    [Fact] public async Task RecursoExistente_ExponeDto()
    {
        using var f=new ApiFactory();
        var id=Guid.NewGuid();
        using(var scope=f.Services.CreateScope())
        {
            var db=scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
            db.Ordenes.Add(new Orden{Id=id,Cliente="ACX-001",Material="CHAPA",Cantidad=10});
            await db.SaveChangesAsync();
        }
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+id);
        var dto=await r.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
        Assert.Equal(id,dto?.Id);
    }
}
```

Por qué funciona así. WebApplicationFactory prueba la aplicación como servidor HTTP: permite distinguir compilación C# de comportamiento observable del enrutamiento y la serialización.

Verificación concreta. dotnet test ejecuta los tres casos con resultado satisfactorio. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Comprobación adicional guiada. Ejecuta ListaVacia_Es200, RecursoInexistente_Es404 y RecursoExistente_ExponeDto. La tercera prueba crea una entidad conocida en la base de test y utiliza HTTP para recuperar su representación; no basta con comprobar que OrdenDto.Desde funciona aisladamente. Si falla únicamente la tercera, revisa el Id usado en la inserción y el GET, la misma base de test y la lectura JSON.

Si algo falla. Si falla ApiFactory, comprueba Program partial y el registro de DbContext con InMemory solo para tests.

PASO 2.1.24 — Reto: incorporar consulta por cliente y explicar su URI

Objetivo del paso. Reto: incorporar consulta por cliente y explicar su URI.

Acciones guiadas. Como reto, acepta ?cliente=ACX-001 en el GET de colección y aplica Where solo cuando se recibe cliente; crea un test.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El filtrado de la colección conserva la idea de recurso y añade una consulta con parámetros, sin transformar el endpoint en una operación RPC.

Verificación concreta. La colección devuelve únicamente órdenes del cliente indicado. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Comprobación adicional guiada. Como ampliación, diseña una consulta por cliente sin alterar las rutas ya aprobadas. Escribe qué ocurriría con un cliente desconocido, cómo se distinguiría una colección vacía de un error y qué test HTTP sería necesario. Si decides implementarla, comprueba que el nuevo endpoint no rompe GET /api/v1/ordenes. El proyecto de referencia de 2.1 no contiene esta extensión y no debe presentarse como una prestación terminada.

Si algo falla. Si el filtro no cambia el resultado, revisa el nombre cliente en la query y la condición Where.

Caso práctico completo — Probar una API de lectura sin depender de una base instalada en CI

Abre el proyecto de pruebas y localiza `ApiFactory.cs`. Esta clase deriva de `WebApplicationFactory<Program>` y arranca una aplicación real de ASP.NET Core en memoria. La aplicación del alumno sigue conectándose a LocalDB cuando se ejecuta en Windows; exclusivamente dentro de la fábrica de tests se sustituye el proveedor por EF Core InMemory. Esa decisión mantiene las pruebas reproducibles y permite que se ejecuten en un runner Linux, pero no autoriza a utilizar InMemory como sustituto de SQL Server para verificar todo su comportamiento.

El primer test consulta la colección vacía y espera 200; el segundo construye un Guid nuevo, lo incorpora a la ruta de detalle y espera 404. Para comprobar el caso positivo se introduce una entidad Orden en la base de la prueba, se confirma SaveChangesAsync y después se envía un GET al mismo identificador. La respuesta se deserializa a OrdenDto. El test debe contrastar al menos el identificador y uno de los campos, porque comprobar solamente un 200 no garantiza que haya aparecido la orden correcta.

Ejecuta desde la terminal integrada `dotnet test` indicando el proyecto `tests/AceriaApp.Api.Tests`. Si un test que insertó una entidad recibe 404, revisa que el mismo contexto de la fábrica comparta base y nombre durante la inserción y el GET. Si el código compila pero el servidor de pruebas no arranca, comprueba que el programa expone `public partial class Program` y que el proyecto de tests referencia la API.

Reto de cierre del punto: escribe una prueba para el filtro opcional por cliente de la colección. Inserta dos órdenes de clientes diferentes, haz la consulta con un parámetro `cliente` y compara los identificadores. La prueba debe rechazar una implementación que devuelva todas las filas.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /api/v1/ordenes`	200 + JSON
`GET /api/v1/ordenes/{id}`	200 / 404

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.2 — Recursos, verbos, persistencia y respuestas HTTP

Proyecto comprobado: `PROYECTO/2.2-recursos-verbos-codigos`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.2.1 — Recuperar el estado GET funcional sin modificarlo

Objetivo del paso. Recuperar el estado GET funcional sin modificarlo.

Acciones guiadas. Abre la etapa 2.1 ya programada; copia tu solución de trabajo a la carpeta del punto 2.2 y ejecuta los GET existentes.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. La lista sigue devolviendo 200 y el detalle inexistente 404.

Comprobación adicional guiada. Abre la etapa 2.1 que ya funcionaba y ejecuta las tres pruebas GET antes de modificarla. Duplica ese estado de referencia para iniciar 2.2. Compara luego Program.cs en ambas carpetas: las dos rutas de lectura deben permanecer disponibles mientras se añaden las operaciones de escritura. Si un GET deja de funcionar después de añadir POST, el cambio se considera una regresión.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.2 — Diseñar la tabla verbo, URI, petición y respuesta

Objetivo del paso. Diseñar la tabla verbo, URI, petición y respuesta.

Acciones guiadas. Construye una tabla con GET/POST/PUT/PATCH/DELETE y las URIs de órdenes; anota cuándo será 201, 204, 400, 404, 409 y 422.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. La tabla coincide con las rutas que programarás, sin acciones Git.

Comprobación adicional guiada. Dibuja una tabla con recurso, método, ruta, cuerpo de entrada y respuesta: POST a colección para crear; PUT y DELETE al Id; PATCH de cantidad al subrecurso /cantidad; POST al Id/finalizar para cambiar el estado. Indica con qué URI se verifica cada cambio mediante GET. Asegúrate de separar claramente 201 con Location, 204 sin cuerpo y 200 con DTO.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.3 — Separar contrato de entrada y DTO de salida

Objetivo del paso. Separar contrato de entrada y DTO de salida.

Acciones guiadas. En la carpeta Contratos dibuja qué acepta el cliente y qué genera el servidor. No recibas Orden directamente desde JSON.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. Id, Estado y FechaCreacion quedan fuera del contrato de creación.

Comprobación adicional guiada. Compara OrdenDto con CrearOrdenRequest y ActualizarCantidadRequest. El DTO de salida contiene Id, Estado y FechaCreacion, mientras que el request de creación solo contiene Cliente, Material y Cantidad. Señala quién decide los campos ausentes al crear el recurso. Este ejercicio previene que se acepte directamente una entidad EF Core enviada desde el exterior.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.4 — Crear CrearOrdenRequest y ActualizarCantidadRequest

Objetivo del paso. Crear CrearOrdenRequest y ActualizarCantidadRequest.

Acciones guiadas. Crea ContratosOrden.cs con CrearOrdenRequest(Cliente, Material, Cantidad) y ActualizarCantidadRequest(Cantidad).

Código real contrastado: **`src/AceriaApp.Api/Contratos/ContratosOrden.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
namespace AceriaApp.Api.Contratos;
// INICIO PASO 2.2.4
public sealed record CrearOrdenRequest(string Cliente,string Material,int Cantidad);
public sealed record ActualizarCantidadRequest(int Cantidad);
// FIN PASO 2.2.4
```

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. Los dos records compilan y se distinguen del OrdenDto de salida.

Comprobación adicional guiada. Crea Contratos/ContratosOrden.cs y declara los dos records, respetando los tipos string e int. Compila antes de modificar Program.cs. En el depurador, un CrearOrdenRequest debe mostrar exactamente tres componentes, no un Guid creado por el cliente. ActualizarCantidadRequest tiene una sola propiedad: no puede cambiar Material mediante el PATCH especializado de esta API.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.5 — Programar MapPost y recibir JSON de una orden

Objetivo del paso. Programar MapPost y recibir JSON de una orden.

Acciones guiadas. En Program.cs declara MapPost /api/v1/ordenes con CrearOrdenRequest y AceriaDbContext. Explica cómo se enlaza JSON.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. La URL acepta POST con Content-Type application/json.

Comprobación adicional guiada. Registra MapPost sobre /api/v1/ordenes con CrearOrdenRequest y AceriaDbContext. Reconoce que el request llega del cuerpo JSON y el contexto se resuelve desde DI. Comprueba que el endpoint está antes de app.Run y que el nombre CrearOrden no reemplaza el nombre ObtenerOrdenPorId del GET. Si falla el enlace del request, revisa Content-Type application/json y la forma del cuerpo.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.6 — Persistir la orden con Add y SaveChangesAsync

Objetivo del paso. Persistir la orden con Add y SaveChangesAsync.

Acciones guiadas. Dentro de POST crea una entidad Orden con los campos del contrato, ejecuta db.Ordenes.Add y SaveChangesAsync.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. Una nueva fila queda persistida en LocalDB.

Comprobación adicional guiada. Crea una instancia Orden a partir de Cliente, Material y Cantidad, utiliza db.Ordenes.Add y llama a await db.SaveChangesAsync. Coloca un breakpoint antes y después de guardar. Explica por qué Add registra el cambio en el contexto, mientras que SaveChangesAsync efectúa la persistencia. No asignes el Estado desde el request: la orden comienza Pendiente según el modelo.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.7 — Devolver 201 y Location con CreatedAtRoute

Objetivo del paso. Devolver 201 y Location con CreatedAtRoute.

Acciones guiadas. Devuelve Results.CreatedAtRoute usando el nombre ObtenerOrdenPorId y el identificador generado.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. El resultado es 201 con cabecera Location.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

Prueba encadenada del 201. Envía POST con Cliente ACX, Material CHAPA y Cantidad 10. Inspecciona la respuesta en tres lugares distintos: el código 201, la cabecera Location y el cuerpo con Id generado. Copia la URI de Location y haz GET. La nueva consulta debe devolver 200 y el mismo Id. Si Location no aparece, revisa el nombre lógico ObtenerOrdenPorId; si el GET no encuentra el recurso, revisa la persistencia. La prueba correspondiente es CrearYConsultar_Es201Y200.

PASO 2.2.8 — Enviar POST y consultar por GET la orden creada

Objetivo del paso. Enviar POST y consultar por GET la orden creada.

Acciones guiadas. Envía POST con ACX-001, CHAPA-3MM y 10; copia el identificador y ejecuta GET de detalle.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. GET muestra los mismos valores guardados por POST.

Comprobación adicional guiada. Después de POST, toma literalmente la cabecera Location devuelta por el servidor y ejecuta GET sin fabricar manualmente un Guid. Compara el Id del recurso consultado con el Id devuelto al crearlo, y verifica Cliente, Material, Cantidad y Estado Pendiente. Este encadenamiento detecta errores de persistencia que un simple Assert del código 201 no descubriría.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.9 — Rechazar provisionalmente peticiones sin datos

Objetivo del paso. Rechazar provisionalmente peticiones sin datos.

Acciones guiadas. En 2.2, antes de introducir FluentValidation, añade un BadRequest provisional ante Cliente o Material vacíos y Cantidad no positiva.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. Un cuerpo incorrecto es distinto de una regla del negocio. En 2.2 el 400 es provisional; 501 unidades reciben 422 por la capacidad limitada del taller.

Verificación concreta. La petición inválida produce 400 y no crea una orden.

Comprobación adicional guiada. Envía un POST con Cliente vacío, otro con Material vacío y uno con Cantidad cero. En la etapa 2.2 los controles son provisionales y se expresan con Results.BadRequest y un objeto de error simple. Anota por qué esas respuestas todavía no tienen necesariamente el formato ValidationProblem que se introducirá con FluentValidation en 2.3.

Si algo falla. Si un 501 da 400, la condición provisional está mezclada con la política de capacidad; distingue ambos casos.

PASO 2.2.10 — Comprobar la política del taller y respuesta 422

Objetivo del paso. Comprobar la política del taller y respuesta 422.

Acciones guiadas. Rechaza Cantidad superior a 500 por política de capacidad con Results.Problem statusCode 422.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. Un cuerpo incorrecto es distinto de una regla del negocio. En 2.2 el 400 es provisional; 501 unidades reciben 422 por la capacidad limitada del taller.

Verificación concreta. El JSON de capacidad 501 produce 422; se distingue de 400.

Comprobación adicional guiada. Envía un POST con Cantidad 501 y observa el 422 producido por la política del taller. Repite con Cantidad 500 y comprueba que esa regla no lo rechaza. No confundas este límite de negocio con el rango genérico 1–10000 que llegará en 2.3. Si el valor 501 llega a persistirse, verifica que el control aparezca antes de db.Ordenes.Add.

Si algo falla. Si un 501 da 400, la condición provisional está mezclada con la política de capacidad; distingue ambos casos.

PASO 2.2.11 — Programar MapPut con identificador Guid

Objetivo del paso. Programar MapPut con identificador Guid.

Acciones guiadas. Declara MapPut /api/v1/ordenes/{id:guid} con Guid, CrearOrdenRequest y AceriaDbContext.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.7: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext
db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")


;
```

Por qué funciona así. PUT sustituye los campos editables de una orden existente. Si el Guid no identifica un recurso debe devolver 404, no crear silenciosamente otro.

Verificación concreta. PUT puede buscar un recurso por identificador.

Comprobación adicional guiada. Declara MapPut sobre /api/v1/ordenes/{id:guid}, recibe el Guid de la ruta y el request del cuerpo. Usa db.Ordenes.FindAsync para localizar la entidad existente. Separa las ramas 404 si falta y 409 si está Finalizada antes de editar. Un PUT sobre una URI arbitraria que no exista no debe convertirse en una creación silenciosa en este ejercicio.

Si algo falla. Si el PUT no actualiza, revisa FindAsync(id) y SaveChangesAsync. El GUID no debe cambiar.

PASO 2.2.12 — Modificar Cliente, Material y Cantidad con PUT

Objetivo del paso. Modificar Cliente, Material y Cantidad con PUT.

Acciones guiadas. En el PUT carga FindAsync(id), devuelve 404 si no existe y sustituye los tres campos editables; SaveChangesAsync.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. PUT sustituye los campos editables de una orden existente. Si el Guid no identifica un recurso debe devolver 404, no crear silenciosamente otro.

Verificación concreta. PUT válido responde 204 y mantiene el mismo Guid.

Comprobación adicional guiada. Asigna al objeto obtenido de EF Core los tres campos editables: Cliente, Material y Cantidad. Mantén Id, Estado y FechaCreacion. Ejecuta la operación con valores distintos a los originales y prepara un GET para comprobarlos. Si accidentalmente se crea una nueva Orden en lugar de modificar la recuperada, estarás cambiando la semántica del PUT esperado.

Si algo falla. Si el PUT no actualiza, revisa FindAsync(id) y SaveChangesAsync. El GUID no debe cambiar.

Caso práctico completo — Una transacción observable: POST, Location, GET y PUT

El primer recorrido de escritura empieza con el método POST. Ejecuta la aplicación y abre `AceriaApp.http`. Envía un cuerpo JSON válido:

```
{"cliente":"ACX-001","material":"CHAPA-3MM","cantidad":10}
```

Utiliza la URI `/api/v1/ordenes` y el verbo POST. La operación construye una entidad nueva de Orden con los datos del contrato. El identificador y la fecha se asignan en el servidor. Cuando `SaveChangesAsync` confirma la inserción, `Results.CreatedAtRoute` prepara un estado 201 con una cabecera `Location`. Abre esa cabecera: no es un mensaje decorativo, sino una ruta que puede usarse después para recuperar la orden.

Envía ahora GET a la URI de `Location` y comprueba que el JSON contiene cliente ACX-001, material CHAPA-3MM y cantidad 10. Guarda el Guid obtenido: lo necesitarás para PUT, PATCH y DELETE. Si GET no encuentra la orden, pregunta qué base de datos usa cada petición y si el POST realizó `SaveChangesAsync`. El hecho de que el POST devuelva un cuerpo con un ID no demuestra por sí solo que se haya insertado una fila.

Para entender PUT, envía a `/api/v1/ordenes/{id}` un cuerpo con cliente ACX-002, material CHAPA-5MM y cantidad 14. El endpoint localiza la orden mediante `FindAsync(id)`, modifica tres propiedades editables y persiste el cambio. El resultado normal es 204, sin un cuerpo de respuesta que copiar. Ejecuta un nuevo GET: el Guid debe ser el mismo, mientras que los tres campos editables reflejan los nuevos valores.

Comprobación: compara lo que cambió y lo que permaneció intacto. El ID, el Estado y la FechaCreacion son valores controlados por el servidor. Si un PUT crea silenciosamente una orden con un Guid inexistente, no está cumpliendo el contrato concreto de este ejemplo.

PASO 2.2.13 — Comprobar 204 y consultar por GET los cambios

Objetivo del paso. Comprobar 204 y consultar por GET los cambios.

Acciones guiadas. Tras PUT ejecuta GET del mismo recurso y comprueba Cliente, Material y Cantidad nuevos.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. PUT sustituye los campos editables de una orden existente. Si el Guid no identifica un recurso debe devolver 404, no crear silenciosamente otro.

Verificación concreta. La consulta evidencia que el cambio se persistió.

Comprobación adicional guiada. Comprueba que PUT devuelve 204 y no exige cuerpo de respuesta. Ejecuta después GET de ese mismo Id y observa los valores actualizados. Diferencia las dos comprobaciones: el estado 204 prueba cómo se respondió, mientras que GET prueba qué estado quedó persistido. Si recibes 200 tras PUT, revisa que no hayas copiado el retorno de PATCH.

Si algo falla. Si el PUT no actualiza, revisa FindAsync(id) y SaveChangesAsync. El GUID no debe cambiar.

PASO 2.2.14 — Programar MapPatch para cambiar solo Cantidad

Objetivo del paso. Programar MapPatch para cambiar solo Cantidad.

Acciones guiadas. Declara MapPatch /api/v1/ordenes/{id:guid}/cantidad con ActualizarCantidadRequest.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.8: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad<=0) return Results.BadRequest(new {error="Cantidad debe ser positiva."});
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")


;
```

Por qué funciona así. PATCH realiza un cambio parcial de Cantidad y conserva los demás datos. Su resultado debe verificarse con una lectura posterior.

Verificación concreta. PATCH admite un cuerpo que contiene solo Cantidad.

Comprobación adicional guiada. Registra MapPatch sobre /api/v1/ordenes/{id:guid}/cantidad, no sobre la colección completa. El cuerpo es ActualizarCantidadRequest y solo contiene Cantidad. Recupera la orden con FindAsync y prepara las ramas 404, 409 y 422 existentes. No utilices JsonPatchDocument: este ejemplo implementa un PATCH especializado, no JSON Patch RFC 6902.

Si algo falla. Si PATCH cambia otros campos, no reasignes Cliente ni Material desde ActualizarCantidadRequest.

PASO 2.2.15 — Comprobar que PATCH no modifica los demás campos

Objetivo del paso. Comprobar que PATCH no modifica los demás campos.

Acciones guiadas. Guarda la nueva cantidad y devuelve Results.Ok(OrdenDto.Desde(orden)); consulta después y verifica que Cliente sigue igual.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. PATCH realiza un cambio parcial de Cantidad y conserva los demás datos. Su resultado debe verificarse con una lectura posterior.

Verificación concreta. Se obtiene 200 y no cambian datos no incluidos en el parche.

Comprobación adicional guiada. Haz GET y anota Cliente y Material; después ejecuta PATCH para cambiar solo Cantidad. Vuelve a hacer GET y compara campo a campo. El resultado normal de PATCH es 200 con DTO, que también permite inspeccionar la nueva cantidad directamente. Si se modifica Cliente por error, revisa qué propiedades toca el delegado.

Si algo falla. Si PATCH cambia otros campos, no reasignes Cliente ni Material desde ActualizarCantidadRequest.

PASO 2.2.16 — Programar MapDelete para eliminar una orden

Objetivo del paso. Programar MapDelete para eliminar una orden.

Acciones guiadas. Declara MapDelete /api/v1/ordenes/{id:guid}; usa FindAsync, Remove y SaveChangesAsync.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.9: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")

;
```

Por qué funciona así. DELETE quita un recurso persistido. La consulta posterior puede devolver 404; esa diferencia no impide que la eliminación sea idempotente.

Verificación concreta. DELETE existente devuelve 204.

Comprobación adicional guiada. Registra MapDelete con Guid, consulta la entidad y verifica si está Finalizada. Solo si sigue pendiente llama a Remove y SaveChangesAsync. El retorno exitoso es NoContent 204. No permitas eliminar una orden finalizada por saltarte la comprobación de Estado; la regla de conflicto forma parte del mismo contrato que posteriormente probará el cliente.

Si algo falla. Si DELETE devuelve éxito pero GET encuentra la orden, revisa Remove y SaveChangesAsync.

PASO 2.2.17 — Verificar 204 seguido de GET 404

Objetivo del paso. Verificar 204 seguido de GET 404.

Acciones guiadas. Llama GET al recurso recién eliminado y después repite DELETE sobre su Id.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. DELETE quita un recurso persistido. La consulta posterior puede devolver 404; esa diferencia no impide que la eliminación sea idempotente.

Verificación concreta. El GET devuelve 404 y el recurso sigue ausente.

Comprobación adicional guiada. Después de DELETE exitoso, envía GET al mismo Id y confirma 404. Vuelve a consultar GET /api/v1/ordenes para comprobar que la orden desapareció de la colección. Un DELETE 204 por sí solo no demuestra la ausencia persistente. Si GET sigue encontrando la orden, revisa la entidad extraída del DbContext y la llamada a SaveChangesAsync.

Si algo falla. Si DELETE devuelve éxito pero GET encuentra la orden, revisa Remove y SaveChangesAsync.

PASO 2.2.18 — Programar la transición de estado finalizar

Objetivo del paso. Programar la transición de estado finalizar.

Acciones guiadas. Declara POST /api/v1/ordenes/{id:guid}/finalizar para establecer Estado=Finalizada y guardar.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.10: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")

;

app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Por qué funciona así. La transición Finalizada cambia el estado de negocio. PUT, PATCH y DELETE deben respetarlo y producir 409 si se intenta una operación incompatible.

Verificación concreta. La transición devuelve 200 y el DTO indica Finalizada.

Comprobación adicional guiada. Añade MapPost a /api/v1/ordenes/{id:guid}/finalizar como transición explícita. Desde una orden Pendiente, asigna Estado Finalizada, guarda y devuelve 200 con OrdenDto. Ejecuta dos veces la misma transición: la segunda debe devolver 409 porque la orden ya estaba Finalizada. Comprueba que este POST actúa sobre el estado de la orden existente, no crea otra fila.

Si algo falla. Si una orden finalizada cambia de estado o desaparece, comprueba la condición Estado antes de escribir.

PASO 2.2.19 — Bloquear PUT y PATCH para órdenes finalizadas

Objetivo del paso. Bloquear PUT y PATCH para órdenes finalizadas.

Acciones guiadas. En PUT y PATCH comprueba Estado=Finalizada antes de modificar y devuelve Problem con 409.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. La transición Finalizada cambia el estado de negocio. PUT, PATCH y DELETE deben respetarlo y producir 409 si se intenta una operación incompatible.

Verificación concreta. Ninguna modificación se guarda sobre una orden finalizada.

Comprobación adicional guiada. Finaliza una orden de prueba y trata de modificarla con PUT y PATCH. Ambos deben producir un conflicto 409. En el depurador observa que la comprobación de Estado sucede antes de modificar Cliente, Material o Cantidad. Consulta luego la orden por GET para verificar que el rechazo no cambió los valores persistidos.

Si algo falla. Si una orden finalizada cambia de estado o desaparece, comprueba la condición Estado antes de escribir.

PASO 2.2.20 — Bloquear DELETE y devolver conflicto 409

Objetivo del paso. Bloquear DELETE y devolver conflicto 409.

Acciones guiadas. En DELETE bloquea también la eliminación de una orden finalizada devolviendo 409.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. La transición Finalizada cambia el estado de negocio. PUT, PATCH y DELETE deben respetarlo y producir 409 si se intenta una operación incompatible.

Verificación concreta. Un recurso finalizado permanece en la base tras DELETE fallido.

Comprobación adicional guiada. Con una orden ya Finalizada, ejecuta DELETE. La respuesta esperada es 409; el recurso debe seguir existiendo y ser consultable con GET. El contraste entre respuesta HTTP y persistencia evita el error de devolver 409 después de haber borrado. Si la orden desaparece, mueve el control de Estado antes de Remove y SaveChangesAsync.

Si algo falla. Si una orden finalizada cambia de estado o desaparece, comprueba la condición Estado antes de escribir.

PASO 2.2.21 — Escribir pruebas HTTP POST y Location

Objetivo del paso. Escribir pruebas HTTP POST y Location.

Acciones guiadas. Añade EscrituraTests con POST, verifica 201, Location y GET posterior del recurso creado.

Código real contrastado: **`tests/AceriaApp.Api.Tests/EscrituraTests.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Modelo;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.2.11: probar POST/GET/PUT/PATCH/DELETE con HTTP.
public class EscrituraTests
{
    [Fact] public async Task CrearYConsultar_Es201Y200()
    {
        using var f=new ApiFactory();using var client=f.CreateClient();
        var created=await client.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        Assert.Equal(HttpStatusCode.Created,created.StatusCode);
        Assert.NotNull(created.Headers.Location);
        var get=await client.GetAsync(created.Headers.Location);
        Assert.Equal(HttpStatusCode.OK,get.StatusCode);
    }
    [Fact] public async Task EditarYEliminar_CambiaPersistencia()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX2","CHAPA",11));
        Assert.Equal(HttpStatusCode.NoContent,put.StatusCode);
        var patch=await c.PatchAsJsonAsync("/api/v1/ordenes/"+dto.Id+"/cantidad",
```

```
            new ActualizarCantidadRequest(12));
        Assert.Equal(HttpStatusCode.OK,patch.StatusCode);
        var changed=await patch.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(12,changed?.Cantidad);
        var del=await c.DeleteAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NoContent,del.StatusCode);
        var gone=await c.GetAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NotFound,gone.StatusCode);
    }
    [Fact] public async Task ReglaCapacidad_Es422()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
    [Fact] public async Task OrdenFinalizada_NoSeEdita409()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",2));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var done=await c.PostAsync("/api/v1/ordenes/"+dto.Id+"/finalizar",null);
        Assert.Equal(HttpStatusCode.OK,done.StatusCode);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX","CHAPA",3));
        Assert.Equal(HttpStatusCode.Conflict,put.StatusCode);
    }
}
```

Por qué funciona así. Los tests deben confirmar los efectos de POST, PUT, PATCH, DELETE, la capacidad del taller y el conflicto de estado a través de peticiones HTTP reales.

Verificación concreta. El test confirma creación y posterior consulta por HTTP.

Comprobación adicional guiada. En EscrituraTests crea una petición mediante PostAsJsonAsync y comprueba HttpStatusCode.Created. Examina created.Headers.Location y utiliza esa dirección en GetAsync. No reemplaces esta prueba de integración por una llamada directa al método que crea la entidad: lo importante es comprobar ruta, serialización, respuesta y Location funcionando conjuntamente.

Si algo falla. Si una prueba de escritura pasa sin volver a consultar la orden, no ha demostrado persistencia; añade el GET posterior.

PASO 2.2.22 — Escribir pruebas HTTP PUT, PATCH y DELETE

Objetivo del paso. Escribir pruebas HTTP PUT, PATCH y DELETE.

Acciones guiadas. En el test del ciclo completo ejecuta PUT, PATCH y DELETE, leyendo el DTO después de cada cambio.

Código real contrastado: `tests/AceriaApp.Api.Tests/EscrituraTests.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Modelo;
using Xunit;
```

```
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.2.11: probar POST/GET/PUT/PATCH/DELETE con HTTP.
public class EscrituraTests
{
    [Fact] public async Task CrearYConsultar_Es201Y200()
    {
        using var f=new ApiFactory();using var client=f.CreateClient();
        var created=await client.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        Assert.Equal(HttpStatusCode.Created,created.StatusCode);
        Assert.NotNull(created.Headers.Location);
        var get=await client.GetAsync(created.Headers.Location);
        Assert.Equal(HttpStatusCode.OK,get.StatusCode);
    }
    [Fact] public async Task EditarYEliminar_CambiaPersistencia()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX2","CHAPA",11));
        Assert.Equal(HttpStatusCode.NoContent,put.StatusCode);
        var patch=await c.PatchAsJsonAsync("/api/v1/ordenes/"+dto.Id+"/cantidad",
            new ActualizarCantidadRequest(12));
        Assert.Equal(HttpStatusCode.OK,patch.StatusCode);
        var changed=await patch.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(12,changed?.Cantidad);
        var del=await c.DeleteAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NoContent,del.StatusCode);
        var gone=await c.GetAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NotFound,gone.StatusCode);
    }
    [Fact] public async Task ReglaCapacidad_Es422()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
    [Fact] public async Task OrdenFinalizada_NoSeEdita409()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",2));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var done=await c.PostAsync("/api/v1/ordenes/"+dto.Id+"/finalizar",null);
        Assert.Equal(HttpStatusCode.OK,done.StatusCode);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX","CHAPA",3));
        Assert.Equal(HttpStatusCode.Conflict,put.StatusCode);
    }
}
```

Por qué funciona así. Los tests deben confirmar los efectos de POST, PUT, PATCH, DELETE, la capacidad del taller y el conflicto de estado a través de peticiones HTTP reales.

Verificación concreta. Se comprueban 204, 200, 204 y después 404.

Comprobación adicional guiada. Construye una prueba que cree una orden, ejecute PUT, ejecute PATCH, lea el DTO actualizado, elimine y finalmente compruebe GET 404. Mantén el mismo Id durante todo el caso y utiliza una factoría aislada para no depender de otras pruebas. Si un paso falla, inspecciona la respuesta de ese paso antes de evaluar el resto del encadenamiento.

Si algo falla. Si una prueba de escritura pasa sin volver a consultar la orden, no ha demostrado persistencia; añade el GET posterior.

PASO 2.2.23 — Probar automáticamente 422 y 409

Objetivo del paso. Probar automáticamente 422 y 409.

Acciones guiadas. Añade pruebas de capacidad 501 y conflicto tras finalizar; ejecuta todo el proyecto xUnit.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. Los tests deben confirmar los efectos de POST, PUT, PATCH, DELETE, la capacidad del taller y el conflicto de estado a través de peticiones HTTP reales.

Verificación concreta. La batería comprueba las respuestas 422 y 409. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Comprobación adicional guiada. Ejecuta los tests ReglaCapacidad_Es422 y OrdenFinalizada_NoSeEdita409. El segundo debe crear una orden y finalizarla antes de pedir su modificación; sin esa preparación no estás reproduciendo el conflicto. Confirma después que las pruebas GET de 2.1 siguen pasando. La nueva funcionalidad no justifica romper el contrato de lectura anterior.

Si algo falla. Si una prueba de escritura pasa sin volver a consultar la orden, no ha demostrado persistencia; añade el GET posterior.

PASO 2.2.24 — Reto: comprobar idempotencia y efecto persistente

Objetivo del paso. Reto: comprobar idempotencia y efecto persistente.

Acciones guiadas. Como reto envía dos PUT idénticos y dos DELETE sobre el mismo Guid. Distingue efecto final de código HTTP.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. La idempotencia habla del efecto final: repetir PUT no multiplica los recursos, y repetir DELETE no restaura lo eliminado. Las respuestas HTTP pueden diferir.

Verificación concreta. El estado del recurso no se duplica y el segundo DELETE no lo restaura. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si se crean órdenes duplicadas al repetir PUT, se está usando Add en lugar de actualizar la entidad encontrada.

Experimento sobre la idempotencia. Crea una orden pendiente y guarda el Id. Ejecuta dos veces el mismo PUT con el mismo Cliente, Material y Cantidad. Verifica mediante GET que repetir la petición no introduce otro cambio en esos campos. Después ejecuta DELETE: la primera respuesta esperada es 204, y un segundo DELETE puede devolver 404 porque el recurso ya no existe. Explica por qué la idempotencia describe el efecto persistente, no que todos los intentos tengan que devolver idéntico código.

Caso práctico completo — PATCH, eliminación e idempotencia: observar el estado final

Conserva la orden creada en el caso anterior. Ejecuta PATCH sobre `/api/v1/ordenes/{id}/cantidad` con el cuerpo:

```
{"cantidad":15}
```

Comprueba que el código es 200 y que la representación devuelta contiene Cantidad=15, Cliente=ACX-002 y Material=CHAPA-5MM. Esto demuestra que PATCH no ha reconstruido el objeto perdiendo otras propiedades. En el método C# correspondiente solo se reasigna `orden.Cantidad`; no se realiza un `new Orden` ni un `db.Ordenes.Add`.

A continuación ejecuta DELETE con el mismo Guid. El servidor localiza la orden, ejecuta `Remove`, confirma `SaveChangesAsync` y devuelve 204. Envía después GET del mismo identificador: debe producir 404. El recurso ya no existe, aunque la ruta como plantilla siga registrada en la API. Si vuelves a ejecutar DELETE, puede responder 404 porque la orden ya no se encuentra. Esto no contradice la idempotencia de DELETE: el estado final sigue siendo ausencia del recurso.

Realiza una segunda secuencia para reproducir conflicto de estado. Crea otra orden, ejecuta POST sobre `/{id}/finalizar` y después intenta actualizarla con PUT o PATCH. La operación debe devolver 409 sin modificar los datos. La política del taller comprueba Estado=Finalizada antes de escribir; el problema no es que el JSON sea inválido, sino que esa transición ya no está permitida. En otra petición distinta envía Cantidad=501: aunque el rango de valores del contrato admite 501, la capacidad del taller está limitada a 500 y devuelve 422.

Reto final: repite exactamente el mismo PUT sobre una orden pendiente dos veces. Comprueba que el cliente, material y cantidad quedan iguales. La idempotencia se entiende observando el efecto final, no contando solamente códigos 204.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`POST /api/v1/ordenes`	201 + Location
`PUT /{id}`	204
`PATCH /{id}/cantidad`	200
`DELETE /{id}`	204 / 404
`POST /{id}/finalizar`	200 / 409
`Cantidad 501`	422

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.3 — Validación efectiva de peticiones con FluentValidation

Proyecto comprobado: `PROYECTO/2.3-validacion-peticiones`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.3.1 — Comprobar endpoints anteriores y localizar sus contratos

Objetivo del paso. Comprobar endpoints anteriores y localizar sus contratos.

Acciones guiadas. Abre el proyecto del punto 2.2 y comprueba GET, POST, PUT, PATCH y DELETE antes de modificarlo.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. La deserialización de un cuerpo JSON y la validación semántica no son lo mismo. Primero el framework interpreta el mensaje y después se comprueban las reglas de datos.

Verificación concreta. Los tests de escritura siguen pasando.

Comprobación adicional guiada. Antes de escribir reglas nuevas, ejecuta los tests heredados de lectura y escritura. Localiza CrearOrdenRequest, ActualizarCantidadRequest y la validación provisional de Program.cs. Señala qué peticiones alcanzan actualmente el delegado y cuáles no. Al terminar 2.3 deberán seguir funcionando los mismos verbos y rutas; cambiaremos el mecanismo de validación, no el modelo de recursos.

Si algo falla. Si JSON roto produce un error inesperado, revisa Content-Type y la estructura del cuerpo antes del validador.

PASO 2.3.2 — Distinguir JSON incorrecto, semántica y negocio

Objetivo del paso. Distinguir JSON incorrecto, semántica y negocio.

Acciones guiadas. Envía tres cuerpos distintos: JSON roto, cantidad 0 y cantidad 501; anota si debe actuar el framework, el validador o el negocio.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. La deserialización de un cuerpo JSON y la validación semántica no son lo mismo. Primero el framework interpreta el mensaje y después se comprueban las reglas de datos.

Verificación concreta. Distingues sintaxis, semántica y regla de capacidad.

Comprobación adicional guiada. Prepara tres cuerpos de petición: JSON mal formado, JSON correcto con Cliente vacío y JSON correcto con Cantidad 501. Predice qué componente detecta cada error: enlazador del framework, validador del request o límite operativo del taller. Conserva esos ejemplos para compararlos tras instalar el filtro. No des por supuesto que todas las respuestas tendrán la misma estructura errors.

Si algo falla. Si JSON roto produce un error inesperado, revisa Content-Type y la estructura del cuerpo antes del validador.

PASO 2.3.3 — Instalar FluentValidation.DependencyInjectionExtensions

Objetivo del paso. Instalar FluentValidation.DependencyInjectionExtensions.

Acciones guiadas. Instala FluentValidation.DependencyInjectionExtensions compatible con .NET 8 en el proyecto de API.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. La deserialización de un cuerpo JSON y la validación semántica no son lo mismo. Primero el framework interpreta el mensaje y después se comprueban las reglas de datos.

Verificación concreta. AbstractValidator y AddValidatorsFromAssemblyContaining se resuelven.

Comprobación adicional guiada. Añade FluentValidation.DependencyInjectionExtensions en la versión compatible con el proyecto de .NET 8. Comprueba que los tipos AbstractValidator, IValidator y AddValidatorsFromAssemblyContaining están disponibles. Restaura y compila antes de conectar el filtro. Instalar un paquete habilita clases y extensiones, pero por sí solo no intercepta ninguna petición HTTP.

Si algo falla. Si JSON roto produce un error inesperado, revisa Content-Type y la estructura del cuerpo antes del validador.

PASO 2.3.4 — Crear CrearOrdenValidator y reglas de Cliente

Objetivo del paso. Crear CrearOrdenValidator y reglas de Cliente.

Acciones guiadas. Crea Validadores.cs con CrearOrdenValidator y reglas RuleFor de Cliente con NotEmpty y MaximumLength(100).

Código real contrastado: `src/AceriaApp.Api/Validacion/Validadores.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.3
public sealed class CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenValidator()
    {
        RuleFor(x=>x.Cliente).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Material).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
    }
}
public sealed class ActualizarCantidadValidator : AbstractValidator<ActualizarCantidadRequest>
{
    public ActualizarCantidadValidator() => RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
}
// FIN PASO 2.3.3
```

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. Un cliente vacío infringe una regla semántica.

Comprobación adicional guiada. En Validacion/Validadores.cs crea CrearOrdenValidator para CrearOrdenRequest. Aplica NotEmpty a Cliente y comprueba que una cadena vacía produce fallo. Añade MaximumLength(100) y prueba mentalmente qué sucede con 100 y 101 caracteres. Distingue el validador de la configuración HasMaxLength(100) en EF Core: el objetivo es devolver un error entendible antes de intentar guardar.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.5 — Definir Material y rango válido de Cantidad

Objetivo del paso. Definir Material y rango válido de Cantidad.

Acciones guiadas. En ese validador incorpora Material obligatorio y Cantidad InclusiveBetween(1,10000).

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. La cantidad 0 o 10001 es inválida; 501 cumple el rango.

Comprobación adicional guiada. Añade NotEmpty y MaximumLength(100) para Material, e InclusiveBetween(1,10000) para Cantidad. Revisa los casos de frontera 0, 1, 500, 501, 10000 y 10001 antes de enviar peticiones. El límite 10000 pertenece al contrato de datos de este taller; la restricción más estricta 500 permanece en el delegado como regla de negocio y producirá 422.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.6 — Crear ActualizarCantidadValidator para PATCH

Objetivo del paso. Crear ActualizarCantidadValidator para PATCH.

Acciones guiadas. Añade ActualizarCantidadValidator para el request de PATCH y su regla de cantidad.

Código real contrastado: `src/AceriaApp.Api/Validacion/Validadores.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.3
public sealed class CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenValidator()
    {
        RuleFor(x=>x.Cliente).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Material).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
    }
}
public sealed class ActualizarCantidadValidator : AbstractValidator<ActualizarCantidadRequest>
{
    public ActualizarCantidadValidator() => RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
}
// FIN PASO 2.3.3
```

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. PATCH comparte el criterio semántico del campo Cantidad.

Comprobación adicional guiada. Crea ActualizarCantidadValidator para ActualizarCantidadRequest con InclusiveBetween(1,10000). No lo sustituyas por CrearOrdenValidator: el PATCH solo recibe Cantidad y no debería pedir Cliente ni Material. Verifica el nombre exacto del record usado por el endpoint y por la regla. Un error de tipo impide conectar el filtro correcto.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.7 — Ejecutar validadores y estudiar ValidationResult

Objetivo del paso. Ejecutar validadores y estudiar ValidationResult.

Acciones guiadas. Instancia los validadores en un test sencillo y observa IsValid y Errors con peticiones válidas e inválidas.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. Comprendes ValidationResult antes de conectar HTTP.

Comprobación adicional guiada. En una pequeña prueba de ejecución o con el depurador, obtén un resultado de ValidateAsync con Cantidad 0 y observa IsValid false y su lista de errores. Repítelo con Cantidad 10 y comprueba IsValid true. La ausencia de excepción no equivale a éxito: se debe inspeccionar ValidationResult. No confundas esta comprobación aislada con una prueba HTTP del endpoint.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.8 — Crear ValidationFilter genérico de Minimal API

Objetivo del paso. Crear ValidationFilter genérico de Minimal API.

Acciones guiadas. Crea ValidationFilter.cs e implementa IEndpointFilter y su InvokeAsync.

Código real contrastado: `src/AceriaApp.Api/Validacion/ValidationFilter.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.4
public sealed class ValidationFilter<T>(IValidator<T> validator): IEndpointFilter where T:class
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var request=context.Arguments.OfType<T>().FirstOrDefault();
        if(request is null) return Results.ValidationProblem(
            new Dictionary<string,string[]> { ["body"]=new[]{"Falta el cuerpo."} });
        var result=await validator.ValidateAsync(request,context.HttpContext.RequestAborted);
        if(!result.IsValid)
        {
            var errors=result.Errors.GroupBy(x=>x.PropertyName)
                .ToDictionary(x=>x.Key,x=>x.Select(e=>e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
        }
        return await next(context);
    }
}
// FIN PASO 2.3.4
```

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. El filtro recibe EndpointFilterInvocationContext y EndpointFilterDelegate.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

Lectura obligatoria antes de ejecutar. Localiza y señala cuatro lugares en ValidationFilter.cs: OfType identifica el argumento, ValidateAsync invoca las reglas, GroupBy/ToDictionary preparan los errores y next(context) permite continuar. Explica por qué llamar a next antes de validar sería peligroso: podría persistirse una orden rechazada. Compilar el filtro no demuestra todavía que se use; su instalación en los endpoints se comprobará en 2.3.15–2.3.18.

PASO 2.3.9 — Obtener la petición de EndpointFilterInvocationContext

Objetivo del paso. Obtener la petición de EndpointFilterInvocationContext.

Acciones guiadas. En InvokeAsync usa context.Arguments.OfType().FirstOrDefault para localizar el contrato de la petición.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. Un request ausente no provoca NullReferenceException.

Comprobación adicional guiada. En InvokeAsync examina context.Arguments y extrae OfType().FirstOrDefault. El argumento esperado debe ser CrearOrdenRequest en POST/PUT y ActualizarCantidadRequest en PATCH. Si no se encuentra, el filtro produce ValidationProblem con clave body. Explica por qué ese caso no implica que cualquier JSON sintácticamente inválido atraviese el filtro: algunos errores suceden durante el enlace previo.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.10 — Ejecutar ValidateAsync y respetar RequestAborted

Objetivo del paso. Ejecutar ValidateAsync y respetar RequestAborted.

Acciones guiadas. Invoca ValidateAsync pasando RequestAborted desde HttpContext para respetar cancelaciones.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. No continúa una operación que el cliente ha cancelado.

Comprobación adicional guiada. Invoca validator.ValidateAsync(request, context.HttpContext.RequestAborted) con await. Localiza RequestAborted en el contexto HTTP y explica para qué sirve si el cliente cancela la solicitud. No llames a SaveChangesAsync dentro del filtro: aquí se decide si permitir la ejecución del manejador, no se persiste el recurso. Compila tras añadir la firma asincrónica.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.11 — Agrupar errores por campo con LINQ

Objetivo del paso. Agrupar errores por campo con LINQ.

Acciones guiadas. Agrupa result.Errors por PropertyName y construye un diccionario string[] de mensajes.

Código real contrastado: **`src/AceriaApp.Api/Validacion/ValidationFilter.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.4
public sealed class ValidationFilter<T>(IValidator<T> validator): IEndpointFilter where T:class
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var request=context.Arguments.OfType<T>().FirstOrDefault();
```

```
        if(request is null) return Results.ValidationProblem(
            new Dictionary<string,string[]> { ["body"]=new[]{"Falta el cuerpo."} });
        var result=await validator.ValidateAsync(request,context.HttpContext.RequestAborted);
        if(!result.IsValid)
        {
            var errors=result.Errors.GroupBy(x=>x.PropertyName)
                .ToDictionary(x=>x.Key,x=>x.Select(e=>e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
        }
        return await next(context);
    }
}
// FIN PASO 2.3.4
```

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. Cada propiedad tiene su colección de errores.

Comprobación adicional guiada. Agrupa result.Errors por PropertyName y convierte cada grupo en un array de ErrorMessage. Para una propiedad con varios errores, no pierdas mensajes sobrescribiendo una cadena con otra. Dibuja una representación con Cliente y Material como claves y una lista de mensajes por clave. Después contrasta la estructura con el objeto que devuelve ValidationProblem.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.12 — Responder mediante Results.ValidationProblem

Objetivo del paso. Responder mediante Results.ValidationProblem.

Acciones guiadas. Ante fallo devuelve Results.ValidationProblem(errors) sin invocar el manejador siguiente.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. La respuesta incluye status 400 y el campo errors.

Comprobación adicional guiada. Utiliza Results.ValidationProblem(errors) cuando IsValid es false. Ejecuta un POST con Cliente vacío tras conectar el filtro y comprueba HTTP 400 y la propiedad JSON errors. No devuelvas Results.Ok con un texto de error: eso haría creer al consumidor que su solicitud fue aceptada. Relaciona el campo errors con el diccionario preparado en el paso anterior.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

Caso práctico completo — Comprobar que el filtro realmente impide entrar al manejador

Arranca el proyecto 2.3 y abre `ValidationFilter.cs` y `Program.cs` al mismo tiempo. Coloca un breakpoint dentro de `InvokeAsync` y otro en la primera instrucción del manejador POST que crea la orden. Prepara una petición con Cliente vacío y los demás campos válidos. El deserializador de ASP.NET Core construirá CrearOrdenRequest porque el JSON tiene una estructura compatible; a continuación se ejecutará el filtro al que se asoció `AddEndpointFilter`.

Inspecciona el valor `request` en el depurador y observa cómo `ValidateAsync` devuelve `IsValid=false`. El filtro agrupa los `ValidationFailure` por nombre de propiedad y pasa el diccionario a `Results.ValidationProblem`. El manejador POST no debe ejecutarse, ni tampoco debe llamarse `SaveChangesAsync`. La respuesta HTTP contiene un 400 y un objeto JSON con la propiedad `errors`.

Repite con un cuerpo válido. El filtro volverá a ejecutarse, pero esta vez `IsValid` será verdadero y se llamará a `await next(context)`. El breakpoint del manejador POST debe alcanzarse. La orden se persiste y el servidor devuelve 201. Esta comparación entre las dos trazas es la demostración más importante del punto: crear una clase validadora y registrarla en DI no basta; hay que conectarla al endpoint que recibe la petición.

Comprueba después PUT y PATCH. El primero utiliza el contrato CrearOrdenRequest y el segundo ActualizarCantidadRequest. Cada uno debe utilizar el tipo genérico correcto al añadir el filtro. Un error común consiste en filtrar solamente POST y creer que todos los verbos quedan automáticamente protegidos. En ese caso PUT o PATCH aceptarían datos que POST rechaza.

Verificación: enviar Cantidad=0 debe devolver 400 con una entrada en errors; Cantidad=501 debe producir 422 porque es una regla del negocio.

PASO 2.3.13 — Permitir next(context) solo cuando IsValid

Objetivo del paso. Permitir next(context) solo cuando IsValid.

Acciones guiadas. Si IsValid es verdadero retorna await next(context); razona qué sucede al omitirlo.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. El endpoint se ejecuta solo cuando la validación pasa.

Comprobación adicional guiada. Coloca return await next(context) exclusivamente en el camino de validación correcta. Comprueba que para Cantidad 0 el filtro termina antes de ejecutar db.Ordenes.Add y que para Cantidad 10 sí entra en el delegado. La diferencia debe observarse en el depurador y posteriormente por HTTP. Si el mismo request crea la orden y devuelve un error, revisa el orden de las instrucciones.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.14 — Registrar validadores en el contenedor DI

Objetivo del paso. Registrar validadores en el contenedor DI.

Acciones guiadas. Antes de builder.Build registra los validadores mediante AddValidatorsFromAssemblyContaining().

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using FluentValidation;
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.5
```

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. El filtro puede resolver IValidator.

Comprobación adicional guiada. Registra builder.Services.AddValidatorsFromAssemblyContaining() antes de builder.Build. Explica que el contenedor encuentra los tipos IValidator correspondientes. Si olvidas este registro, la construcción de ValidationFilter no podrá resolver correctamente su dependencia. No confundas registrar validadores con instalarlos automáticamente en todos los endpoints.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.15 — Conectar filtro de CrearOrdenRequest al POST

Objetivo del paso. Conectar filtro de CrearOrdenRequest al POST.

Acciones guiadas. Añade AddEndpointFilter<ValidationFilter>() al MapPost real.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
.WithName("CrearOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()

;
```

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. POST con Cliente vacío devuelve 400 en vez de guardar una fila.

Comprobación adicional guiada. En el MapPost de órdenes añade AddEndpointFilter<ValidationFilter> después del registro de ruta. Ejecuta una petición con Cliente vacío y verifica 400 con errors; luego una válida y verifica 201 con Location. Estas dos pruebas son indispensables: la primera demuestra que el filtro corta, y la segunda que no bloquea indiscriminadamente todas las peticiones.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.16 — Conectar filtro de CrearOrdenRequest al PUT

Objetivo del paso. Conectar filtro de CrearOrdenRequest al PUT.

Acciones guiadas. Añade el mismo filtro de CrearOrdenRequest al MapPut.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. PUT inválido no modifica la orden almacenada.

Comprobación adicional guiada. Aplica el filtro del mismo tipo CrearOrdenRequest al MapPut. Usa una orden Pendiente creada previamente para comprobar un PUT con datos válidos y otro con Material vacío. No pruebes solo un Id inexistente: el manejador podría devolver 404 y no demostrarías que el filtro actúa. Verifica después por GET que el intento inválido no alteró los campos guardados.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.17 — Conectar filtro de ActualizarCantidadRequest al PATCH

Objetivo del paso. Conectar filtro de ActualizarCantidadRequest al PATCH.

Acciones guiadas. Añade AddEndpointFilter<ValidationFilter>() al MapPatch.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
.WithName("CambiarCantidad").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
```

```
;
```

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. PATCH con Cantidad=0 devuelve 400.

Comprobación adicional guiada. Aplica ValidationFilter al MapPatch de cantidad. Prueba Cantidad 0 frente a Cantidad 12. En el primer caso espera 400; en el segundo 200 con DTO actualizado, siempre que la orden permanezca Pendiente. Confirma que el filtro no exige Cliente ni Material, ya que no forman parte del contrato de este PATCH.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.18 — Enviar POST incorrecto y comprobar status 400

Objetivo del paso. Enviar POST incorrecto y comprobar status 400.

Acciones guiadas. Envía un POST con Material vacío y comprueba código, Content-Type y ausencia de inserción.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. El código es 400 y la base permanece sin cambios.

Comprobación adicional guiada. En AceriaApp.http configura Content-Type application/json y envía POST con Cliente vacío. Registra estado 400 y el cuerpo; no te quedes únicamente con el texto visible del mensaje. Ejecuta después la misma solicitud con Cliente ACX y comprueba 201. Si ambos casos obtienen 201, vuelve a Program.cs y confirma que AddEndpointFilter está realmente encadenado al MapPost.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.19 — Inspeccionar la propiedad errors del Problem Details

Objetivo del paso. Inspeccionar la propiedad errors del Problem Details.

Acciones guiadas. Examina el JSON Problem Details y localiza en errors el nombre de la propiedad rechazada.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. El cliente puede mostrar el mensaje bajo el campo correcto.

Comprobación adicional guiada. Abre la respuesta 400 y examina la propiedad errors; identifica la clave Cliente o su representación serializada y el array de mensajes. Compara esa estructura con un 422 emitido por Results.Problem: ambos son errores, pero no tienen por qué publicar exactamente las mismas claves. Evita hacer tests que dependan de la redacción exacta de mensajes localizados.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.20 — Probar límite 10001 frente a la capacidad 501

Objetivo del paso. Probar límite 10001 frente a la capacidad 501.

Acciones guiadas. Compara un POST de cantidad 10001 con otro de 501 y anota el origen del error.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. El primero devuelve 400, el segundo 422.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

Dos valores, dos responsabilidades. Envía POST con Cliente ACX, Material CHAPA y Cantidad 10001; debe devolver 400 con errors por superar el rango del validador. Repite con Cantidad 501; debe devolver 422 por exceder el máximo operativo del taller y no debe crear la orden. Localiza en Validadores.cs el límite genérico y en Program.cs la regla de negocio. Si ambas respuestas pasan a ser 400, comprueba que alguien no haya reducido por error el rango genérico hasta 500.

PASO 2.3.21 — Reproducir 409 con orden ya finalizada

Objetivo del paso. Reproducir 409 con orden ya finalizada.

Acciones guiadas. Crea, finaliza e intenta editar una orden: no es un fallo semántico, sino un conflicto de estado.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. Se recibe 409 y los datos no cambian.

Comprobación adicional guiada. Crea una orden válida, ejecuta POST /api/v1/ordenes/{id}/finalizar y después intenta editar mediante PUT. El conflicto 409 debe producirse dentro del delegado por Estado Finalizada, no en FluentValidation. Consulta el recurso por GET tras el intento: los datos no deben haber cambiado. Si obtienes 400, verifica que el request PUT era válido antes de probar la regla de estado.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.22 — Crear pruebas de validación mediante HTTP

Objetivo del paso. Crear pruebas de validación mediante HTTP.

Acciones guiadas. Crea ValidacionHttpTests con peticiones reales para Cliente, Material, Cantidad y política de negocio.

Código real contrastado: **`tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using AceriaApp.Api.Contratos;
using Xunit;
namespace AceriaApp.Api.Tests;
```

```
// INICIO PASO 2.3.8: se verifica el filtro ACTIVO, no su mera existencia.
public class ValidacionHttpTests
{
    [Theory]
    [InlineData("","CHAPA",10)]
    [InlineData("ACX","",10)]
    [InlineData("ACX","CHAPA",0)]
    public async Task DatosInvalidos_Es400ConErrors(string cliente,string material,int cantidad)
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest(cliente,material,cantidad));
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        Assert.True(json.RootElement.TryGetProperty("errors",out _));
    }
    [Fact] public async Task CantidadMasDe10000_Es400()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10001));
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
    }
    [Fact] public async Task ReglaDeNegocio_501_Es422()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
}
```

Por qué funciona así. Las pruebas HTTP recorren el pipeline completo y comprueban que los fallos se comunican como Problem Details sin alterar los datos.

Verificación concreta. Las respuestas se prueban a través del pipeline completo.

Comprobación adicional guiada. Escribe ValidacionHttpTests con PostAsJsonAsync para Cliente vacío, Material vacío y Cantidad cero, y comprueba BadRequest y la propiedad errors. Añade los casos 10001 y 501 para separar 400 de 422. Cada test debe crear su propia ApiFactory; así no dependerá del orden de ejecución ni de datos guardados por otro método de prueba.

Si algo falla. Si el test solo instancia un record, no prueba HTTP; usa WebApplicationFactory y examina errors del JSON.

PASO 2.3.23 — Verificar que los GET siguen sin regresiones

Objetivo del paso. Verificar que los GET siguen sin regresiones.

Acciones guiadas. Ejecuta los tests GET y de escritura anteriores, además de las nuevas pruebas de validación.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. Las pruebas HTTP recorren el pipeline completo y comprueban que los fallos se comunican como Problem Details sin alterar los datos.

Verificación concreta. El filtrado no rompe las rutas que ya funcionaban. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Comprobación adicional guiada. Ejecuta las pruebas GET, las de escritura y las nuevas de validación dentro de la etapa 2.3. Los tres casos GET originales deben seguir funcionando: un filtro nuevo sobre POST/PUT/PATCH no debe cambiar las rutas GET. Si aparece una regresión, localiza el primer cambio que alteró el endpoint anterior en vez de borrar la prueba heredada.

Si algo falla. Si el test solo instancia un record, no prueba HTTP; usa WebApplicationFactory y examina errors del JSON.

PASO 2.3.24 — Reto: personalizar mensajes y comprobar errores

Objetivo del paso. Reto: personalizar mensajes y comprobar errores.

Acciones guiadas. Reto: incorpora WithMessage a una regla y comprueba desde un test HTTP el texto dentro de errors.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. Las pruebas HTTP recorren el pipeline completo y comprueban que los fallos se comunican como Problem Details sin alterar los datos.

Verificación concreta. El mensaje devuelto es específico y mantiene status 400. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Comprobación adicional guiada. Personaliza un mensaje del validador como ampliación y envía una petición que incumpla exactamente esa regla. Comprueba que el campo afectado conserva su clave y que solo cambia el mensaje esperado. Repite un caso válido para asegurarte de no bloquear la creación normal. La solución de referencia no incluye esta personalización, por lo que debes distinguir claramente el reto de lo implementado.

Si algo falla. Si el test solo instancia un record, no prueba HTTP; usa WebApplicationFactory y examina errors del JSON.

Caso práctico completo — Del error 400 al 409: tres pruebas con causas diferentes

Para comprobar la separación de responsabilidades, prepara primero un POST de cantidad 10.001. Es un número entero válido desde el punto de vista sintáctico, pero infringe la regla `InclusiveBetween(1,10000)`. Por tanto, FluentValidation genera un error semántico y el filtro devuelve 400. Inspecciona el campo `errors`: debe identificar Cantidad. No necesitas consultar SQL Server para determinar que un valor está fuera de ese intervalo.

Envía después exactamente el mismo cuerpo, cambiando Cantidad a 501. El validador semántico deja pasar el valor, porque 501 pertenece al rango de entrada. El manejador POST comprueba entonces la capacidad del taller y decide responder 422. La decisión podría depender en el futuro de inventario, estado de las máquinas o política comercial; por esa razón no es idéntica a una comprobación de formato. Este es un contrato elegido para el ejercicio, no una regla universal que obligue a todas las APIs a usar 422.

La tercera prueba necesita un recurso persistido. Crea una orden con cantidad 10, recupera su Guid, ejecútale `/finalizar` y después envía un PUT de modificación. El servidor devuelve 409 porque el recurso está en un estado incompatible con la modificación. Un 409 no significa que el tipo `int` de Cantidad sea inválido: significa que el recurso existente no admite la operación solicitada.

Escribe pruebas en `ValidacionHttpTests.cs` que cubran las tres situaciones. Usa `WebApplicationFactory` para hacer peticiones HTTP reales y comprobar el código que recibe un consumidor. La prueba de 400 debe revisar `errors`; las de 422 y 409 comprueban la distinción de política y estado.

Control de regresión: después de añadir filtros, vuelve a ejecutar GET de lista y detalle. Ninguna regla de validación debería convertir una lectura válida en un fallo de contrato.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`Cliente vacío`	400 + errors
`Cantidad 10001`	400 + errors
`Cantidad 501`	422
`Orden finalizada`	409

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.4 — Documento OpenAPI generado y comprobado con Swagger

Proyecto comprobado: `PROYECTO/2.4-documentacion-openapi`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.4.1 — Comprobar el contrato REST que se va a documentar

Objetivo del paso. Comprobar el contrato REST que se va a documentar.

Acciones guiadas. Abre la etapa 2.3 y anota sus rutas, DTOs, estados 200/201/204/400/409/422 antes de documentarla.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.4-documentacion-openapi` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. La futura documentación reflejará endpoints ya comprobados.

Comprobación adicional guiada. Ejecuta los tests del estado 2.3 y enumera las rutas efectivas de Program.cs: GET colección, GET individual, POST colección, PUT por Id, PATCH de cantidad, DELETE por Id y POST finalizar. Clasifica sus respuestas normales y alternativas. Esta lista, derivada del código y no de un esquema inventado, será la referencia para revisar lo que posteriormente anuncie Swagger.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.2 — Instalar Swashbuckle.AspNetCore compatible con .NET 8

Objetivo del paso. Instalar Swashbuckle.AspNetCore compatible con .NET 8.

Acciones guiadas. Instala Swashbuckle.AspNetCore 6.6.2 para .NET 8 en el proyecto API, no solo en tests.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.4-documentacion-openapi` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. AddSwaggerGen y UseSwagger están disponibles.

Comprobación adicional guiada. Instala Swashbuckle.AspNetCore compatible con .NET 8 y restaura paquetes. Comprueba que el proyecto reconoce AddSwaggerGen, UseSwagger y UseSwaggerUI. No sustituyas automáticamente el mecanismo por bibliotecas de versiones posteriores de .NET: este ejercicio muestra el paquete elegido. Una restauración correcta confirma dependencias, no que el JSON OpenAPI tenga rutas correctas.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.3 — Crear carpeta Documentacion y extensiones de configuración

Objetivo del paso. Crear carpeta Documentacion y extensiones de configuración.

Acciones guiadas. Crea Documentacion/DocumentacionExtensions.cs y una clase estática DocumentacionExtensions.

Código real contrastado: `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
        return app;
    }
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
```

```
            ["cantidad"]=new OpenApiInteger(10)
        };
    }
}
// FIN PASO 2.4.5
```

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. La configuración OpenAPI queda separada de Program.cs.

Comprobación adicional guiada. Crea Documentacion/DocumentacionExtensions.cs y define una clase estática. Separa AddDocumentacion, que registra servicios, de UseDocumentacion, que instala exposición HTTP. Comprueba los namespaces de Microsoft.OpenApi y de Swashbuckle. No mezcles en ese archivo las reglas de CrearOrdenValidator: la documentación debe describir el contrato sin asumir la responsabilidad de validar peticiones.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.4 — Registrar AddEndpointsApiExplorer y AddSwaggerGen

Objetivo del paso. Registrar AddEndpointsApiExplorer y AddSwaggerGen.

Acciones guiadas. En AddDocumentacion registra AddEndpointsApiExplorer y AddSwaggerGen con opciones.

Código real contrastado: `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
        return app;
    }
```

```
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
            ["cantidad"]=new OpenApiInteger(10)
        };
    }
}
// FIN PASO 2.4.5
```

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. Los servicios descubren los endpoints Minimal API.

Comprobación adicional guiada. Dentro de AddDocumentacion incorpora services.AddEndpointsApiExplorer y services.AddSwaggerGen. Antes de iniciar el host, localiza qué endpoints Minimal API se han declarado y qué metadatos podrán descubrirse. Si Swagger muestra un documento vacío, comprueba tanto este registro como la existencia real de MapGet y MapPost, antes de añadir rutas manuales al JSON.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.5 — Configurar SwaggerDoc con información y versión v1

Objetivo del paso. Configurar SwaggerDoc con información y versión v1.

Acciones guiadas. Configura SwaggerDoc v1 con título, versión, descripción y contacto de AceriaApp.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. La sección info del JSON muestra esos valores.

Comprobación adicional guiada. Configura SwaggerDoc para v1, título AceriaApp — Órdenes de producción y descripción educativa. Verifica que en el JSON info.version aparece v1 y que los nombres de documentación no cambian las rutas /api/v1/ordenes. Identifica la diferencia entre el nombre de documento v1 y la cadena literal de versión que aparece en cada URI.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.6 — Conectar AddDocumentacion en Program.cs

Objetivo del paso. Conectar AddDocumentacion en Program.cs.

Acciones guiadas. En Program.cs llama builder.Services.AddDocumentacion() antes de builder.Build().

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
builder.Services.AddDocumentacion(); // PASO 2.4.3
```

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. Se registra el generador en el contenedor.

Comprobación adicional guiada. Invoca builder.Services.AddDocumentacion antes de builder.Build. Compila y confirma que el método de extensión se resuelve mediante el namespace AceriaApp.Api.Documentacion. En este punto aún falta instalar el middleware de Swagger; el registro de servicios y la disponibilidad pública del documento son responsabilidades distintas.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.7 — Configurar UseSwagger y Swagger UI en Development

Objetivo del paso. Configurar UseSwagger y Swagger UI en Development.

Acciones guiadas. En UseDocumentacion coloca UseSwagger y UseSwaggerUI dentro de la condición IsDevelopment.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. Swagger no se publica automáticamente en Production.

Comprobación adicional guiada. En UseDocumentacion añade un control IsDevelopment y, dentro de él, app.UseSwagger y app.UseSwaggerUI. Establece SwaggerEndpoint con /swagger/v1/swagger.json. No coloques UseSwagger fuera de esa condición: el test posterior exige que el documento no se publique desde un host Production. Si cambia la URL del JSON, sincroniza la configuración de la UI y las pruebas.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.8 — Conectar UseDocumentacion y abrir /swagger

Objetivo del paso. Conectar UseDocumentacion y abrir /swagger.

Acciones guiadas. Llama app.UseDocumentacion() después de construir la aplicación y abre /swagger desde Visual Studio.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
app.UseDocumentacion(); // PASO 2.4.4
```

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. La interfaz presenta los endpoints del proyecto.

Comprobación adicional guiada. Invoca app.UseDocumentacion antes de app.Run, arranca en Development y abre /swagger. Si la UI carga pero no muestra operaciones, abre directamente el JSON del endpoint configurado y comprueba si la ausencia está en el documento o en la interfaz. La página de Swagger no es una respuesta creada por ninguno de los delegados que persisten órdenes.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.9 — Descargar el JSON real /swagger/v1/swagger.json

Objetivo del paso. Descargar el JSON real /swagger/v1/swagger.json.

Acciones guiadas. Descarga /swagger/v1/swagger.json como texto JSON, no una cadena YAML fabricada.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. La petición devuelve 200 y el documento empieza por openapi.

Comprobación adicional guiada. Solicita GET /swagger/v1/swagger.json mediante el navegador o el archivo HTTP y guarda la respuesta JSON real. Revisa que el estado sea 200 en Development. No pegues un documento OpenAPI ficticio en el proyecto como reemplazo de la generación: la finalidad es demostrar que el documento deriva de los endpoints implementados.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.10 — Identificar info, paths, components y schemas

Objetivo del paso. Identificar info, paths, components y schemas.

Acciones guiadas. Busca las secciones info, paths y components; identifica las rutas de órdenes y sus verbos.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. GET, POST, PUT, PATCH y DELETE figuran donde corresponde.

Comprobación adicional guiada. Identifica en el JSON las propiedades info, paths y components.schemas. Busca /api/v1/ordenes en paths y CrearOrdenRequest en los esquemas. Explica cuál describe metadatos de la API, cuál la superficie de rutas y cuál la forma de los mensajes. Si una propiedad no aparece, inspecciona el registro del generador y las anotaciones de los endpoints.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.11 — Aplicar WithName y WithTags a los endpoints

Objetivo del paso. Aplicar WithName y WithTags a los endpoints.

Acciones guiadas. En Program.cs examina WithName y WithTags para agrupar los endpoints de órdenes.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. Los nombres se utilizan también en CreatedAtRoute.

Comprobación adicional guiada. Asigna nombres distintos mediante WithName y agrupa mediante WithTags("Ordenes"). Comprueba que los nombres lógicos ListarOrdenes, ObtenerOrdenPorId y CrearOrden no sustituyen las URIs públicas. Si escribes por error dos nombres iguales, revisa además las referencias que CreatedAtRoute utiliza para construir Location; la documentación no debe romper la navegación HTTP.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.12 — Documentar GET con WithSummary y Produces

Objetivo del paso. Documentar GET con WithSummary y Produces.

Acciones guiadas. Agrega WithSummary y Produces a los GET y comprueba 200 y 404.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
```

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. El documento describe la lectura y la ausencia del recurso.

Comprobación adicional guiada. Añade WithSummary y Produces<IEnumerable>(200) al GET de colección. Para GET individual anuncia también 404, porque la rama Results.NotFound existe. Abre Swagger JSON y compara el resumen y las responses de ambos GET. Produce metadatos, no comportamiento: quita mentalmente Produces y observa que Results.Ok seguiría devolviendo 200.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

Caso práctico completo — Desde MapGet hasta paths: interpretar Swagger JSON

Inicia la API 2.4 en Development y abre la URL `http://localhost:5082/swagger/v1/swagger.json`. Este documento no se guarda a mano en la carpeta docs. Lo produce SwaggerGen cuando la aplicación solicita información sobre los endpoints de ASP.NET Core. Busca la propiedad `info`: debe contener el título de AceriaApp y la versión v1 que registró `SwaggerDoc`.

Continúa con la sección `paths`. Busca `/api/v1/ordenes` y despliega sus operaciones `get` y `post`. Cada operación representa un manejador declarado mediante MapGet o MapPost. La sección muestra los tipos de cuerpo de entrada y las respuestas descritas con `Produces`, `ProducesProblem` y otros metadatos. Una ruta ausente en el documento puede indicar que no está registrada, que la exploración no está configurada o que sus metadatos son incompletos. No lo arregles escribiendo una ruta ficticia en un YAML separado.

Abre a continuación Swagger UI en `/swagger`. La interfaz interpreta el JSON y permite visualizar los endpoints agrupados con `WithTags`. Selecciona GET de colección, ejecuta Try it out y compara el estado 200 con el del archivo HTTP del alumno. Las dos herramientas consumen la misma API; Swagger UI no proporciona un servidor de ejemplo independiente.

En `Program.cs` revisa `WithSummary` y `Produces`. Añade o corrige el resumen del GET y vuelve a descargar el JSON: el metadato cambia porque la fuente de la documentación es el código. Esta relación bidireccional entre implementar y documentar es precisamente el aprendizaje del punto 2.4.

Error habitual: configurar AddSwaggerGen pero olvidar `app.UseDocumentacion()` después de Build. En ese caso los servicios pueden estar registrados y, sin embargo, /swagger devolver 404.

PASO 2.4.13 — Documentar POST con 201 y ValidationProblem 400

Objetivo del paso. Documentar POST con 201 y ValidationProblem 400.

Acciones guiadas. Documenta POST con Produces(201) y ProducesValidationProblem(400).

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. Swagger enumera creación y error de contrato.

Comprobación adicional guiada. Documenta POST con 201, ValidationProblem 400 y Problem 422; conserva CreatedAtRoute dentro del manejador. Descarga el JSON y busca las respuestas de la operación post. Si la documentación anunciase 200 como éxito único, no coincidiría con el comportamiento de creación real. Comprueba después un POST desde el cliente HTTP para verificar que 201 sigue operativo.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.14 — Documentar PUT y PATCH con códigos alternativos

Objetivo del paso. Documentar PUT y PATCH con códigos alternativos.

Acciones guiadas. Documenta PUT y PATCH con sus códigos, evitando declarar respuestas inventadas.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. La documentación debe coincidir con los Results usados.

Comprobación adicional guiada. Anota en PUT los posibles 204, 404, 400, 409 y 422; en PATCH de cantidad, 200, 404, 400, 409 y 422. Verifica cada estado frente a las ramas de Program.cs. En la etapa 2.4 actualizada se incluyen metadatos explícitos para esas alternativas. No asumas que ProducesProblem agrega automáticamente el código de fallo a los delegados: describe la respuesta, no la ejecuta.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.15 — Documentar DELETE y el conflicto 409

Objetivo del paso. Documentar DELETE y el conflicto 409.

Acciones guiadas. Documenta DELETE y el conflicto 409 de órdenes finalizadas.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. El consumidor puede anticipar una eliminación rechazada.

Comprobación adicional guiada. Documenta DELETE con 204, 404 y conflicto 409. Comprueba que 204 sigue sin cuerpo y que 409 se produce cuando la orden ya está Finalizada. Revisa también la acción POST finalizar, que puede devolver 200, 404 o 409. La documentación debe expresar las ramas existentes, no códigos genéricos copiados de una lista teórica de HTTP.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.16 — Crear SchemaFilter EjemploCrearOrden

Objetivo del paso. Crear SchemaFilter EjemploCrearOrden.

Acciones guiadas. Crea EjemploCrearOrden : ISchemaFilter y asigna OpenApiObject para los tres campos del request.

Código real contrastado: **`src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
        return app;
    }
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
            ["cantidad"]=new OpenApiInteger(10)
        };
    }
}
// FIN PASO 2.4.5
```

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. El esquema contiene cliente, material y cantidad ilustrativos.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

Separación entre ejemplo y transacción. Abre /swagger/v1/swagger.json en Development y localiza components.schemas.CrearOrdenRequest.example. Verifica que el ejemplo incluya ACX-001, CHAPA-3MM y Cantidad 10. Este documento no guarda una orden en la base de datos. Solamente al ejecutar de verdad el POST desde Swagger UI se produce la creación; anota ambas pruebas en columnas distintas para no confundir documentación y funcionamiento.

PASO 2.4.17 — Inspeccionar example de CrearOrdenRequest en Swagger

Objetivo del paso. Inspeccionar example de CrearOrdenRequest en Swagger.

Acciones guiadas. En JSON comprueba components.schemas.CrearOrdenRequest.example y explica que no son datos persistidos.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. El ejemplo aparece en la sección reusable de esquemas.

Comprobación adicional guiada. Solicita el JSON y abre components.schemas.CrearOrdenRequest. Identifica example, los tipos de cliente, material y cantidad, y relaciónalo con el record de entrada. Comprueba que el ejemplo no pertenece a OrdenDto, porque éste es el contrato de salida. Si el objeto example falta, revisa que SchemaFilter se registró y que context.Type coincide con CrearOrdenRequest.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.18 — Ejecutar un POST desde Swagger UI

Objetivo del paso. Ejecutar un POST desde Swagger UI.

Acciones guiadas. En Swagger UI utiliza Try it out para enviar POST de orden y observar 201 y Location.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. Se crea un recurso de verdad desde el explorador HTTP.

Comprobación adicional guiada. En Swagger UI expande POST /api/v1/ordenes, habilita la edición del cuerpo y utiliza el ejemplo de entrada con Cantidad 10. Ejecuta y observa el estado, la respuesta y la cabecera Location. Recuerda que la interfaz está realizando una solicitud real: si utilizas LocalDB de desarrollo, la orden queda persistida como en cualquier otro POST.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.19 — Consultar la orden creada desde la propia UI

Objetivo del paso. Consultar la orden creada desde la propia UI.

Acciones guiadas. Copia el GUID obtenido y ejecuta GET de detalle desde la UI.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. GET responde 200 con el DTO de la misma orden.

Comprobación adicional guiada. Toma el Id del resultado de POST y ejecuta el GET individual desde la misma UI. Compara Cliente, Material, Cantidad y Estado. Si el POST dijo 201 pero el GET no encuentra el recurso, investiga persistencia y generación de Location, no solo los esquemas de Swagger. La utilidad de una UI interactiva es precisamente contrastar documentación con ejecución.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.20 — Crear tests HTTP del documento OpenAPI real

Objetivo del paso. Crear tests HTTP del documento OpenAPI real.

Acciones guiadas. En ContratoOpenApiTests, usa WebApplicationFactory para solicitar el JSON real.

Código real contrastado: `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Text.Json;
using Microsoft.AspNetCore.Hosting;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.4.8: abrir OpenAPI JSON REAL mediante HTTP.
public class ContratoOpenApiTests
{
    [Fact] public async Task OpenApi_ContieneRutasYVersion()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());

Assert.Equal("v1",json.RootElement.GetProperty("info").GetProperty("version").GetString());
        var paths=json.RootElement.GetProperty("paths");
        Assert.True(paths.TryGetProperty("/api/v1/ordenes",out var resource));
        Assert.True(resource.TryGetProperty("get",out _));
        Assert.True(resource.TryGetProperty("post",out _));
    }
    [Fact] public async Task OpenApi_IncluyeEjemplo()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        var schemas=json.RootElement.GetProperty("components").GetProperty("schemas");
        Assert.True(schemas.TryGetProperty("CrearOrdenRequest",out var schema));
        Assert.True(schema.TryGetProperty("example",out _));
    }
    [Fact] public async Task Produccion_NoExponeSwagger()
    {
        using var f=new ApiFactory();
        using var other=f.WithWebHostBuilder(b=>b.UseEnvironment("Production"));
        var r=await other.CreateClient().GetAsync("/swagger/v1/swagger.json");
        Assert.Equal(HttpStatusCode.NotFound,r.StatusCode);
    }
}
```

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. El test descarga la respuesta del servidor y la analiza.

Comprobación adicional guiada. Crea ContratoOpenApiTests usando WebApplicationFactory y HttpClient. En el test solicita realmente /swagger/v1/swagger.json y analiza el resultado con JsonDocument. No leas un archivo de muestra incluido en el proyecto. Esta prueba debe ejercitar AddDocumentacion, UseDocumentacion, el enrutamiento del documento y el generador de Swashbuckle.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.21 — Comprobar rutas GET y POST en paths

Objetivo del paso. Comprobar rutas GET y POST en paths.

Acciones guiadas. Comprueba info.version y rutas de paths con operaciones get y post.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. Un cambio de ruta erróneo hace fallar el contrato.

Comprobación adicional guiada. En el JSON de pruebas busca paths./api/v1/ordenes y comprueba que contiene los miembros get y post. Mantén las comprobaciones dentro del host real de test. Añade como ampliación las rutas PUT, PATCH y DELETE solo si también actualizas la prueba y el código de metadatos; la nueva prueba OpenApi_DeclaraErroresDeOperacionesReales comprueba respuestas 409 y 422 publicadas.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.22 — Comprobar schema y ejemplo de petición

Objetivo del paso. Comprobar schema y ejemplo de petición.

Acciones guiadas. Agrega un test que compruebe CrearOrdenRequest y su propiedad example en components.schemas.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. El esquema se genera a partir del contrato público.

Comprobación adicional guiada. Comprueba que components.schemas contiene CrearOrdenRequest y que el esquema declara example. Relaciona las claves camelCase del ejemplo con los nombres de las propiedades del record. No compares todos los caracteres del documento como si la posición de las propiedades JSON fuera relevante: lo importante es la estructura y contenido contractual.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.23 — Comprobar que Production responde 404 para Swagger

Objetivo del paso. Comprobar que Production responde 404 para Swagger.

Acciones guiadas. Ejecuta el test en Environment Production y solicita /swagger/v1/swagger.json.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. El endpoint no se publica y responde 404. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

Comparación reproducible entre entornos. Solicita /swagger/v1/swagger.json en Development y confirma 200. Ejecuta después Produccion_NoExponeSwagger de ContratoOpenApiTests.cs: crea un host de pruebas con Environment Production y la misma ruta debe devolver 404. Comprueba la condición

IsDevelopment en UseDocumentacion. La ausencia de Swagger en Production no prueba por sí sola que los endpoints estén autenticados o protegidos; no atribuyas al ejemplo garantías que todavía no implementa.

PASO 2.4.24 — Reto: documentar PATCH y verificar su summary en JSON

Objetivo del paso. Reto: documentar PATCH y verificar su summary en JSON.

Acciones guiadas. Reto: añade un resumen específico a PATCH y comprueba que paths muestra summary actualizado.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.4-documentacion-openapi` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Por qué funciona así. Cambiar un WithSummary y ver reflejada la modificación en el JSON demuestra la vinculación de la documentación con el código ejecutado.

Verificación concreta. Se verifica que el documento evoluciona al modificar el código. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Comprobación adicional guiada. Como reto, cambia WithSummary del PATCH por una frase más precisa y vuelve a consultar el JSON OpenAPI. Localiza la operación patch de la ruta /api/v1/ordenes/{id}/cantidad y verifica que la propiedad summary refleja tu modificación. No cambies la ruta ni el request para conseguirlo. Repite los tests de documentación y los de ejecución para descartar regresiones.

Si algo falla. Si el summary no cambia, modifica WithSummary en el endpoint real y vuelve a solicitar el documento.

Caso práctico completo — Contrato de ejemplo y prueba en Production

La sección `components.schemas` documenta objetos que pueden reutilizarse desde varias operaciones. Busca `CrearOrdenRequest`. Debe exponer las propiedades cliente, material y cantidad. El `SchemaFilter EjemploCrearOrden` incorpora valores orientativos para esos campos; no crea ninguna orden. El propósito es ayudar a quien consume el API a preparar un cuerpo JSON compatible.

Desde Swagger UI abre el POST de órdenes y utiliza Try it out. Envía un JSON con ACX-001, CHAPA-3MM y cantidad 10; el servidor debe devolver 201 y Location. Guarda el identificador y solicita GET de detalle desde la misma interfaz. Si obtienes una respuesta distinta, revisa el cuerpo, las reglas de validación y la conexión a la base de datos; no concluyas que el contrato está bien porque Swagger haya dibujado el formulario.

Para probar la documentación automáticamente, abre `ContratoOpenApiTests.cs`. La prueba arranca WebApplicationFactory, solicita el JSON real y analiza sus propiedades con JsonDocument. Comprueba `info.version`, las rutas de `paths` y el esquema de CrearOrdenRequest con su `example`. La prueba debe fallar si el endpoint POST se elimina sin actualizar el contrato de la API.

Finalmente crea la fábrica de tests con `UseEnvironment("Production")` y vuelve a solicitar `/swagger/v1/swagger.json`. El método `UseDocumentacion` restringe la publicación a Development; por eso la respuesta esperada es 404. Esta comprobación enseña a separar la presencia de un paquete de su exposición pública.

Reto final: modifica `WithSummary` de PATCH, arranca de nuevo y comprueba en el JSON de `paths` que el resumen se actualizó. No escribas un documento OpenAPI manual: la prueba debe demostrar que la documentación sigue derivándose de la implementación.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /swagger/v1/swagger.json`	200 en Development
`info.version`	v1

Comportamiento	Resultado esperado
`paths contiene GET y POST`	Rutas descubiertas
`schemas contiene CrearOrdenRequest`	Contrato registrado
`GET /swagger/v1/swagger.json`	404 en Production

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.
